

# Contents

|          |                                                                                   |          |
|----------|-----------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Chapter 1: What Is Runact?</b>                                                 | <b>1</b> |
| 1.1      | The Actor Model . . . . .                                                         | 1        |
| 1.2      | Why Rust? . . . . .                                                               | 1        |
| 1.3      | BEAM Inspiration . . . . .                                                        | 1        |
| 1.4      | Runact at a Glance . . . . .                                                      | 2        |
| 1.5      | Design Principles . . . . .                                                       | 2        |
| 1.6      | Quick Start . . . . .                                                             | 3        |
| 1.7      | 2.1 Actor . . . . .                                                               | 3        |
| 1.8      | 2.2 ActorId . . . . .                                                             | 4        |
|          | 1.8.1 Addressing Without IDs . . . . .                                            | 4        |
| 1.9      | 2.3 Mailbox . . . . .                                                             | 4        |
|          | 1.9.1 Bounded Mailboxes and Backpressure . . . . .                                | 4        |
|          | 1.9.2 Reduction Counting . . . . .                                                | 5        |
| 1.10     | 2.4 Runtime . . . . .                                                             | 5        |
|          | 1.10.1 RuntimeSender . . . . .                                                    | 5        |
| 1.11     | 2.5 Supervision . . . . .                                                         | 5        |
| 1.12     | 2.6 Async Tasks vs. Actors . . . . .                                              | 6        |
| 1.13     | 2.7 The Runtime Architecture . . . . .                                            | 6        |
| 1.14     | 3.1 Project Setup . . . . .                                                       | 6        |
| 1.15     | 3.2 The Complete Example . . . . .                                                | 7        |
| 1.16     | 3.3 Line-by-Line Walkthrough . . . . .                                            | 8        |
|          | 1.16.1 Imports . . . . .                                                          | 8        |
|          | 1.16.2 The Actor . . . . .                                                        | 8        |
|          | 1.16.3 Implementing Actor . . . . .                                               | 8        |
|          | 1.16.4 The Handle Method . . . . .                                                | 8        |
|          | 1.16.5 Creating the Runtime . . . . .                                             | 9        |
|          | 1.16.6 Spawning . . . . .                                                         | 9        |
|          | 1.16.7 Sending a Message . . . . .                                                | 9        |
|          | 1.16.8 Shutting Down . . . . .                                                    | 9        |
| 1.17     | 3.4 What Happens When You Run It . . . . .                                        | 9        |
| 1.18     | 3.5 Making It Do Work . . . . .                                                   | 9        |
| 1.19     | 4.1 The Trait . . . . .                                                           | 11       |
|          | 1.19.1 Send + 'static (trait bound) . . . . .                                     | 11       |
|          | 1.19.2 type Message: Send + 'static . . . . .                                     | 11       |
|          | 1.19.3 fn handle(&mut self, message, context) -> Result<(), ActorError> . . . . . | 11       |
| 1.20     | 4.2 Returning Errors . . . . .                                                    | 11       |

|      |                                                     |    |
|------|-----------------------------------------------------|----|
| 1.21 | 4.3 A Complete Example: A Counter Actor . . . . .   | 12 |
| 1.22 | 4.4 Stateful Actors . . . . .                       | 13 |
| 1.23 | 4.5 Multi-Message-Type Actors . . . . .             | 14 |
| 1.24 | 4.6 Actor Lifecycle . . . . .                       | 14 |
| 1.25 | 5.1 The Context API . . . . .                       | 15 |
| 1.26 | 5.2 Sending Messages to Other Actors . . . . .      | 15 |
|      | 1.26.1 send_to vs runtime.send . . . . .            | 16 |
| 1.27 | 5.3 Reply-to-Sender (Request/Reply) . . . . .       | 16 |
| 1.28 | 5.4 Scheduling Timers . . . . .                     | 17 |
|      | 1.28.1 One-shot Timer . . . . .                     | 17 |
|      | 1.28.2 Periodic Timer (Interval) . . . . .          | 17 |
|      | 1.28.3 Use Case: Heartbeat . . . . .                | 17 |
| 1.29 | 5.5 Offloading CPU-Bound Work . . . . .             | 18 |
| 1.30 | 5.6 The actor_id() Method . . . . .                 | 18 |
| 1.31 | 5.7 Important: Handle Must Return Quickly . . . . . | 19 |
| 1.32 | 6.1 What Is a Mailbox? . . . . .                    | 19 |
| 1.33 | 6.2 Bounded Mailboxes . . . . .                     | 19 |
| 1.34 | 6.3 Backpressure . . . . .                          | 20 |
|      | 1.34.1 Handling MailboxFull . . . . .               | 20 |
|      | 1.34.2 send vs send_blocking . . . . .              | 20 |
| 1.35 | 6.4 Reduction Counting . . . . .                    | 20 |
|      | 1.35.1 How Reductions Work . . . . .                | 20 |
|      | 1.35.2 Why It Matters . . . . .                     | 21 |
| 1.36 | 6.5 Idle Actors . . . . .                           | 21 |
| 1.37 | 6.6 Mailbox Internals . . . . .                     | 21 |
| 1.38 | 7.1 Why Supervision? . . . . .                      | 22 |
| 1.39 | 7.2 RestartStrategy . . . . .                       | 22 |
|      | 1.39.1 OneForOne . . . . .                          | 22 |
|      | 1.39.2 Parameters . . . . .                         | 22 |
|      | 1.39.3 Default Strategy . . . . .                   | 22 |
| 1.40 | 7.3 ChildSpec . . . . .                             | 22 |
|      | 1.40.1 RestartPolicy . . . . .                      | 23 |
|      | 1.40.2 Builder Pattern . . . . .                    | 23 |
| 1.41 | 7.4 Spawning a Supervisor . . . . .                 | 23 |
| 1.42 | 7.5 How Restarts Work . . . . .                     | 23 |
|      | 1.42.1 Exponential Backoff . . . . .                | 24 |
| 1.43 | 7.6 Supervised Worker Example . . . . .             | 24 |
| 1.44 | 7.7 Supervisor as an Actor . . . . .                | 24 |
| 1.45 | 7.8 Key Principles . . . . .                        | 24 |
| 1.46 | 8.1 Message Types . . . . .                         | 25 |
| 1.47 | 8.2 Fire-and-Forget: send_to . . . . .              | 25 |
|      | 1.47.1 From Within an Actor . . . . .               | 25 |
|      | 1.47.2 From External Code . . . . .                 | 26 |
| 1.48 | 8.3 Request/Reply: Runtime::request . . . . .       | 26 |
|      | 1.48.1 RequestHandle Methods . . . . .              | 27 |
| 1.49 | 8.4 Actor-to-Actor Request/Reply . . . . .          | 27 |
| 1.50 | 8.5 ActorId as a Communication Primitive . . . . .  | 28 |
| 1.51 | 8.6 Message Ordering . . . . .                      | 28 |

|          |        |                                             |           |
|----------|--------|---------------------------------------------|-----------|
| 1.52     | 9.1    | spawn_task . . . . .                        | 29        |
|          | 1.52.1 | Key Guarantees . . . . .                    | 29        |
| 1.53     | 9.2    | TaskHandle . . . . .                        | 29        |
|          | 1.53.1 | recv — Block Until Done . . . . .           | 29        |
|          | 1.53.2 | try_recv — Check Without Blocking . . . . . | 29        |
|          | 1.53.3 | recv_timeout — Block With Timeout . . . . . | 30        |
|          | 1.53.4 | Output Must Be Clone . . . . .              | 30        |
| 1.54     | 9.3    | Async Sleep and Timeout . . . . .           | 30        |
| 1.55     | 9.4    | Bridging Actors and Async Tasks . . . . .   | 30        |
| 1.56     | 9.5    | Task Groups and Cancellation . . . . .      | 31        |
| 1.57     | 10.1   | One-shot Timer . . . . .                    | 32        |
| 1.58     | 10.2   | Periodic Timer (Interval) . . . . .         | 32        |
|          | 1.58.1 | Message Requirements . . . . .              | 33        |
|          | 1.58.2 | Returns TimerId . . . . .                   | 33        |
| 1.59     | 10.3   | TimerId and Cancellation . . . . .          | 33        |
| 1.60     | 10.4   | Drift Correction . . . . .                  | 34        |
| 1.61     | 10.5   | Runtime-Level Timers . . . . .              | 34        |
| 1.62     | 10.6   | Use Cases . . . . .                         | 34        |
| <b>2</b> |        | <b>Chapter 11: Compute Pool</b>             | <b>35</b> |
| 2.1      | 11.1   | Why a Separate Pool? . . . . .              | 35        |
| 2.2      | 11.2   | ctx.spawn_compute . . . . .                 | 35        |
|          | 2.2.1  | Requirements . . . . .                      | 36        |
| 2.3      | 11.3   | ComputeHandle . . . . .                     | 36        |
|          | 2.3.1  | Polling for Results . . . . .               | 36        |
| 2.4      | 11.4   | Cancellation . . . . .                      | 36        |
| 2.5      | 11.5   | ComputeConfig . . . . .                     | 37        |
|          | 2.5.1  | Queue Capacity and Backpressure . . . . .   | 37        |
| 2.6      | 11.6   | Use Cases for Compute Pool . . . . .        | 37        |
| 2.7      | 11.7   | ComputePool vs AsyncTask . . . . .          | 38        |
| 2.8      | 12.1   | Design Philosophy . . . . .                 | 38        |
|          | 2.8.1  | Threading Model . . . . .                   | 38        |
| 2.9      | 12.2   | TcpListener . . . . .                       | 38        |
|          | 2.9.1  | accept() . . . . .                          | 39        |
|          | 2.9.2  | local_addr() . . . . .                      | 39        |
| 2.10     | 12.3   | TcpStream . . . . .                         | 39        |
|          | 2.10.1 | Read . . . . .                              | 39        |
|          | 2.10.2 | write_all + flush . . . . .                 | 39        |
|          | 2.10.3 | read_exact . . . . .                        | 39        |
|          | 2.10.4 | read_to_end . . . . .                       | 39        |
|          | 2.10.5 | shutdown . . . . .                          | 39        |
| 2.11     | 12.4   | Simple Echo Server . . . . .                | 40        |
| 2.12     | 12.5   | TCP with Actors . . . . .                   | 40        |
| 2.13     | 12.6   | I/O Reactor . . . . .                       | 41        |
| 2.14     | 13.1   | The Request Type . . . . .                  | 41        |
|          | 2.14.1 | Parsing from Bytes . . . . .                | 42        |
|          | 2.14.2 | Path Parameters . . . . .                   | 42        |
| 2.15     | 13.2   | The Response Type . . . . .                 | 42        |

|          |                                      |           |
|----------|--------------------------------------|-----------|
| 2.15.1   | Encoding                             | 42        |
| 2.16     | 13.3 The Router                      | 42        |
| 2.16.1   | Router::handle                       | 43        |
| 2.17     | 13.4 Handler Trait                   | 43        |
| 2.18     | 13.5 Building a Complete HTTP Server | 44        |
| 2.19     | 13.6 IntoResponse                    | 44        |
| <b>3</b> | <b>Chapter 14: WebSocket Server</b>  | <b>47</b> |
| 3.1      | 14.1 WebSocketServer                 | 47        |
| 3.1.1    | ServerEvent                          | 48        |
| 3.1.2    | WebSocketConfig                      | 48        |
| 3.2      | 14.2 Stream Abstraction              | 48        |
| 3.2.1    | Standard TCP                         | 48        |
| 3.2.2    | Runact TCP Bridge                    | 48        |
| 3.3      | 14.3 Frame API                       | 48        |
| 3.3.1    | OpCode                               | 49        |
| 3.4      | 14.4 ConnectionWriter                | 49        |
| 3.4.1    | SendError                            | 49        |
| 3.5      | 14.5 Echo Server Example             | 49        |
| 3.6      | 14.6 Fragmentation Handling          | 50        |
| 3.7      | 14.7 Handshake                       | 50        |
| 3.8      | 15.1 Fragmentation Overview          | 50        |
| 3.8.1    | Rules (RFC 6455 §5.4)                | 51        |
| 3.9      | 15.2 Runact's Reassembly             | 51        |
| 3.9.1    | Key Design Points                    | 52        |
| 3.10     | 15.3 Multi-Frame read() Processing   | 52        |
| 3.10.1   | frame_encoded_size                   | 52        |
| 3.11     | 15.4 Testing Fragmentation           | 53        |
| 3.12     | 16.1 Architecture                    | 53        |
| 3.13     | 16.2 The Agent Message Protocol      | 53        |
| 3.14     | 16.3 The ModelAdapter Trait          | 54        |
| 3.15     | 16.4 The AgentActor                  | 54        |
| 3.16     | 16.5 Bridging WebSocket to Actors    | 55        |
| 3.16.1   | Key Patterns                         | 55        |
| 3.17     | 16.6 Full Example                    | 55        |
| 3.18     | 16.7 Testing                         | 56        |
| 3.19     | 16.8 Production Considerations       | 56        |
| 3.20     | 17.1 Architecture                    | 56        |
| 3.21     | 17.2 The Router                      | 57        |
| 3.21.1   | Path Parameters                      | 57        |
| 3.22     | 17.3 Shared State                    | 58        |
| 3.23     | 17.4 Reading HTTP Requests           | 58        |
| 3.24     | 17.5 The Full Server Loop            | 58        |
| 3.24.1   | Why Thread-Per-Connection?           | 59        |
| 3.25     | 17.6 JSON Helpers                    | 59        |
| 3.26     | 17.7 Testing                         | 60        |
| 3.27     | 18.1 runact::Runtime                 | 60        |
| 3.28     | 18.2 runact::RuntimeSender           | 61        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 3.29     | 18.3 Actor Types . . . . .                               | 61        |
| 3.30     | 18.4 Error Types . . . . .                               | 62        |
| 3.31     | 18.5 Config Types . . . . .                              | 62        |
| 3.32     | 18.6 runact-web Public API . . . . .                     | 63        |
| 3.33     | 18.7 Task Types . . . . .                                | 63        |
| <b>4</b> | <b>Chapter 19: Invariants</b>                            | <b>65</b> |
| 4.1      | 19.1 Async Runtime Invariants . . . . .                  | 65        |
| 4.2      | 19.2 Networking Invariants . . . . .                     | 65        |
| 4.3      | 19.3 Hard Rules (from AGENTS.md) . . . . .               | 66        |
| 4.4      | 19.4 Verification . . . . .                              | 66        |
| <b>5</b> | <b>Chapter 20: Architectural Decisions</b>               | <b>67</b> |
| 5.1      | 20.1 ADR-0001: BEAM-Style Scheduler . . . . .            | 67        |
| 5.2      | 20.2 ADR-0002: Dual Scheduler Architecture . . . . .     | 67        |
| 5.3      | 20.3 ADR-0003: Native Async Runtime (No Tokio) . . . . . | 67        |
| 5.4      | 20.4 Additional Decisions . . . . .                      | 68        |
| 5.4.1    | Compute Pool Separation . . . . .                        | 68        |
| 5.4.2    | Capability-Based Security . . . . .                      | 68        |
| 5.4.3    | Cooperative Cancellation . . . . .                       | 68        |



# Chapter 1

## Chapter 1: What Is Runact?

### 1.1 The Actor Model

The **actor model** is a way to structure concurrent computation. It was originally developed by Carl Hewitt in 1973 and later popularized by Erlang (the BEAM virtual machine). In the actor model:

- **Every actor is an independent unit of computation.** It has its own state, isolated from all other actors.
- **Actors communicate exclusively by passing messages.** You never touch another actor's memory directly.
- **An actor decides what to do with each message:** send messages to other actors, spawn new actors, change its own behavior (state), or schedule a future action.

This gives you **location transparency** (an actor doesn't need to know whether its message target runs on the same machine or a remote one) and **fault isolation** (a crash in one actor doesn't corrupt another actor's memory).

### 1.2 Why Rust?

Rust brings two key advantages to the actor model:

1. **Ownership guarantees:** Because actors never share mutable memory, Rust's borrow checker aligns perfectly with actor isolation. Messages are moved (not shared), so the compiler can prove at compile time that no aliasing or data races exist.
2. **Zero-cost abstractions:** Runact's actors and schedulers compile down to efficient machine code. There is no garbage collector, no VM overhead.

### 1.3 BEAM Inspiration

Runact's scheduler is inspired by the BEAM (Erlang VM) scheduler:

Scheduler  
(work-stealing)

P1          P2          PN      ← Worker threads  
                                          ← Run queues

Each worker thread maintains a run queue. When a worker runs out of work, it **steals** work from another worker’s queue. Actors (called “processes” in BEAM terms) are extremely lightweight — a single Runact actor can cost as little as a few hundred bytes of heap.

## 1.4 Runact at a Glance

| Concept      | Runact Type                                       | Purpose                                     |
|--------------|---------------------------------------------------|---------------------------------------------|
| Actor        | <code>impl Actor</code>                           | Isolated unit of state + behavior           |
| Message      | <code>Actor::Message</code>                       | Data sent between actors                    |
| ActorId      | <code>ActorId</code>                              | Unique address of an actor                  |
| Runtime      | <code>Runtime</code>                              | Top-level coordinator                       |
| Scheduler    | <code>Scheduler</code> (private)                  | Assigns actors to worker threads            |
| Supervision  | <code>Supervisor</code>                           | Fault-tolerant child management             |
| Async tasks  | <code>spawn</code> , <code>TaskHandle</code>      | Standard Rust <code>Future</code> execution |
| Compute pool | <code>ComputeHandle</code>                        | CPU-bound work off the scheduler            |
| Timers       | <code>schedule_timer</code>                       | Delayed and periodic message delivery       |
| TCP          | <code>TcpListener</code> , <code>TcpStream</code> | Native async I/O readiness                  |

## 1.5 Design Principles

1. **Runact owns concurrency and task lifecycle.** It manages actors, async tasks, the scheduler, and I/O readiness.
2. **Higher-level libraries own protocols.** HTTP, WebSocket, TLS, DNS, and application-level protocols are implemented in separate crates (`runact-web` for HTTP/WebSocket). Runact core has no these dependencies.
3. **No Tokio in the core.** Runact implements its own executor. Tokio integration is possible at the application layer if needed, but it is not fundamental to Runact’s operation.
4. **Actors are synchronous and cooperative.** An actor’s `handle` method runs to completion for each message. It must return quickly — no blocking.
5. **CPU-heavy work never runs on actor or async workers.** It goes to the dedicated compute pool.
6. **Cancellation is cooperative.** Tasks and actors can be told to stop, but they must check for cancellation themselves.

## 1.6 Quick Start

Here is the complete “Hello, World” in Runact:

```
use runact::{Actor, ActorContext, ActorError, Runtime};

struct Greeter;

impl Actor for Greeter {
    type Message = String;

    fn handle(
        &mut self,
        msg: String,
        ctx: &mut ActorContext,
    ) -> Result<(), ActorError> {
        println!("Hello, {}!", msg);
        Ok(())
    }
}

fn main() {
    let mut runtime = Runtime::new().unwrap();
    let id = runtime.spawn(Greeter).unwrap();
    runtime.send(id, "world".to_string()).unwrap();
    runtime.shutdown();
}
```

This creates a runtime, spawns one actor, sends it a message, and shuts down. Chapter 3 walks through this example line by line. # Chapter 2: Core Concepts

Before writing your first actor, you need to understand five foundational concepts that Runact builds on. This chapter explains each one and how they fit together.

---

## 1.7 2.1 Actor

An **actor** is a self-contained unit of state and behavior. Each actor:

- Owns its private state (fields on the struct).
- Defines a single message type it accepts.
- Implements the **Actor** trait, which provides a **handle** method called for every incoming message.

```
pub trait Actor: Send + 'static {
    type Message: Send + 'static;

    fn handle(
        &mut self,
```

```

        message: Self::Message,
        context: &mut ActorContext,
    ) -> Result<(), ActorError>;
}

```

**Key rules:** - An actor's handle must be **synchronous** and **non-blocking**. Do not call `std::thread::sleep`, do not block on I/O, do not loop forever. - When an actor receives a message, it processes that message to completion before moving to the next one. - Actors never share mutable memory. Communication is exclusively through messages, which are **moved** (not borrowed).

## 1.8 2.2 ActorId

Every spawned actor is assigned an `ActorId` — a unique identifier within the runtime. You use it to address the actor when sending messages.

```

let id: ActorId = runtime.spawn(MyActor).unwrap();
runtime.send(id, some_message()).unwrap();

```

`ActorId` is: - Clone — you can hand copies to other actors. - Copy — it is just a u64 wrapper. - Debug, Hash, Eq — usable as a key in `HashMap`.

### 1.8.1 Addressing Without IDs

If an actor needs to send messages to another actor regularly, store the `ActorId` as a field:

```

struct Client {
    server: ActorId,
}

impl Actor for Client {
    type Message = String;
    fn handle(&mut self, msg: String, ctx: &mut ActorContext) -> Result<(), ActorError> {
        // Forward the message to the server
        ctx.send_to(self.server, msg)?;
        Ok(())
    }
}

```

## 1.9 2.3 Mailbox

Each actor has a **mailbox** — a bounded queue of incoming messages. When you call `runtime.send(id, msg)`, the message is pushed onto the actor's mailbox. The scheduler picks up the actor when it has messages waiting and polling budget remaining.

### 1.9.1 Bounded Mailboxes and Backpressure

By default, Runact uses a **bounded mailbox** (capacity configurable via `RuntimeConfig`). When the mailbox is full:

Sender send() Mailbox (full) reject Caller gets `RuntimeError::MailboxFull`

This is **backpressure**: the sender is forced to slow down when the receiver can't keep up. This prevents unbounded memory growth.

If you need fire-and-forget with backpressure, check for `RuntimeError::MailboxFull` and retry with a delay, or drop the message gracefully.

### 1.9.2 Reduction Counting

Each actor processes a bounded number of messages per scheduling quantum (a “reduction” budget, similar to BEAM’s reduction counting). When the budget is exhausted, the actor yields, and another actor gets scheduled. This ensures fair scheduling — no single actor can monopolize a worker thread.

## 1.10 2.4 Runtime

The **Runtime** is the top-level coordinator. It owns:

- The **scheduler** (assigns actors to worker threads)
- The **compute pool** (offloads CPU-bound work)
- The **task executor** (runs async `Futures`)
- The **timer service** (schedules delayed/periodic messages)
- The **actor registry** (maps `ActorId` → mailbox senders)

You create a runtime with `Runtime::new()` and tear it down with `runtime.shutdown()`.

```
let mut runtime = Runtime::new().unwrap();
// ... spawn actors, send messages ...
runtime.shutdown();
```

### 1.10.1 RuntimeSender

For sending messages from **outside** the actor system (e.g., from a callback, a thread, or an async task), use `RuntimeSender`:

```
let sender = runtime.sender();
sender.send(actor_id, message).unwrap();
```

`RuntimeSender` is `Clone`, so you can give it to any thread or async task that needs to message an actor.

## 1.11 2.5 Supervision

**Supervision** is Runact’s fault-tolerance mechanism. A **Supervisor** spawns child actors and defines what happens when a child panics:

| Strategy                | On child panic                      |
|-------------------------|-------------------------------------|
| <code>OneForOne</code>  | Restart only the child              |
| <code>RestForOne</code> | Restart child + subsequent siblings |

Supervisors use **exponential backoff**: after each restart, the delay doubles (up to a cap). If a child crashes too many times within a window, the supervisor gives up and the whole subtree is torn down.

Supervision ties together with the actor lifecycle: a child actor receives an `ActorContext::init` call on startup (including restarts), so it can re-initialize its state cleanly.

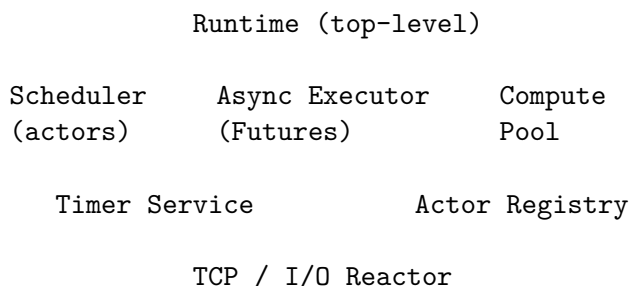
## 1.12 2.6 Async Tasks vs. Actors

Runact has two concurrency primitives:

| Feature          | Actor                                                 | Async Task                                     |
|------------------|-------------------------------------------------------|------------------------------------------------|
| Concurrency unit | <code>Actor</code> trait impl                         | <code>Future</code>                            |
| Communication    | Messages ( <code>send</code> , <code>request</code> ) | Return value ( <code>TaskHandle::recv</code> ) |
| Scheduling       | Reductions (cooperative)                              | Task budget (cooperative)                      |
| Error handling   | Panic → supervisor restart                            | <code>Result&lt;T, TaskError&gt;</code>        |
| Use case         | Stateful services, long-lived                         | Short-lived I/O, one-shot work                 |

**Rule of thumb:** If your unit of work has identity and long-lived state, use an **actor**. If it's a short-lived computation (e.g., an HTTP request handler), use an **async task**.

## 1.13 2.7 The Runtime Architecture



The scheduler and async executor share worker threads but use **reduction budgeting** to ensure fairness between actors and async tasks. CPU-bound work is offloaded to the compute pool so it never blocks a worker. # Chapter 3: Your First Actor

This chapter walks through a complete, runnable Runact program — from `Cargo.toml` to shutdown — and explains each line.

## 1.14 3.1 Project Setup

```
# Cargo.toml
[package]
```

```
name = "runact-hello"
version = "0.1.0"
edition = "2024"

[dependencies]
runact = { path = "../runact" }
```

**Note:** In this book, code examples use the local path to the runact crate. When publishing, use the crates.io version.

## 1.15 3.2 The Complete Example

```
use runact::{Actor, ActorContext, ActorError, Runtime};

/// A simple actor that prints a greeting.
struct Greeter;

impl Actor for Greeter {
    type Message = String;

    fn handle(
        &mut self,
        msg: String,
        _ctx: &mut ActorContext,
    ) -> Result<(), ActorError> {
        println!("Hello, {}!", msg);
        Ok(())
    }
}

fn main() {
    // 1. Create the runtime
    let mut runtime = Runtime::new().unwrap();

    // 2. Spawn the actor
    let id = runtime.spawn(Greeter).unwrap();

    // 3. Send a message
    runtime.send(id, "world".to_string()).unwrap();

    // 4. Shut down
    runtime.shutdown();
}
```

## 1.16 3.3 Line-by-Line Walkthrough

### 1.16.1 Imports

```
use runact::{Actor, ActorContext, ActorError, Runtime};
```

Four imports cover the whole public API surface of the core types:

| Import       | Role                                   |
|--------------|----------------------------------------|
| Actor        | The trait you implement                |
| ActorContext | Methods for messaging, timers, compute |
| ActorError   | Return type for error handling         |
| Runtime      | Top-level coordinator                  |

### 1.16.2 The Actor

```
struct Greeter;
```

An actor is just a normal Rust struct. You can add fields for state:

```
struct Counter {
    count: u64,
}
```

The struct must be `Send + 'static` (guaranteed by the `Actor` trait bound). This means no `&T` references to non-`Send` data.

### 1.16.3 Implementing Actor

```
impl Actor for Greeter {
    type Message = String;
```

The `type Message = String` declares what kind of message this actor accepts. Any type works as long as it is `Send + 'static` — `String`, structs, enums, tuples, or even `()`.

### 1.16.4 The Handle Method

```
fn handle(
    &mut self,
    msg: String,
    _ctx: &mut ActorContext,
) -> Result<(), ActorError> {
    println!("Hello, {}!", msg);
    Ok(())
}
```

- `&mut self` — you own the actor's state for the duration of this call. Mutate fields freely.
- `msg: String` — the message, **moved** into the handler. No copying, no aliasing.
- `_ctx: &mut ActorContext` — use this to send messages to other actors, spawn timers, submit compute work, or spawn tasks. We prefix with `_` because this actor doesn't need it yet.

- `Result<(), ActorError>` — return `Ok()` on success. Return `Err(ActorError::...)` if the message couldn't be processed; the supervisor decides what to do (typically restart the actor).

### 1.16.5 Creating the Runtime

```
let mut runtime = Runtime::new().unwrap();
```

`Runtime::new()` sets up the scheduler, compute pool, timer service, and task executor with default configuration. It returns `Result<Runtime, RuntimeError>` — the `unwrap()` is fine for examples, but production code should handle the error.

### 1.16.6 Spawning

```
let id = runtime.spawn(Greeter).unwrap();
```

`spawn` takes ownership of the actor, assigns it an `ActorId`, and starts polling it. The actor won't process messages until the scheduler gets to it — but since we send a message right after spawning, the scheduler will pick it up in the next cycle.

### 1.16.7 Sending a Message

```
runtime.send(id, "world".to_string()).unwrap();
```

`send` is fire-and-forget. The message is pushed onto the actor's mailbox. If the mailbox is full (bounded, default capacity), `send` returns `RuntimeError::MailboxFull`. `send` takes `mut self` — it needs `&mut` to the runtime because it interacts with the scheduler's registry.

### 1.16.8 Shutting Down

```
runtime.shutdown();
```

`shutdown` signals all actors, async tasks, timers, and compute workers to stop. It blocks until they finish or the shutdown timeout expires. After `shutdown`, no new actors can be spawned and no new messages can be sent.

## 1.17 3.4 What Happens When You Run It

1. `Runtime::new()` starts worker threads (one per CPU core by default).
2. `spawn(Greeter)` creates the actor, assigns it `ActorId(1)`, and pushes it onto a worker's run queue.
3. `send(id, "world")` pushes the message onto the actor's mailbox. Since the actor now has a pending message, the scheduler ensures a worker will poll it.
4. The scheduler worker picks up the actor, calls `handle`, and prints `Hello, world!`.
5. `shutdown()` drains all pending work and joins worker threads.

## 1.18 3.5 Making It Do Work

The `Greeter` above is trivial — it prints and exits. Real actors use `ActorContext` to interact with the runtime. Here is a slightly more advanced example that uses a timer and sends to itself:

```

use runact::{Actor, ActorContext, ActorError, Runtime};
use std::time::Duration;

struct Ticker {
    remaining: u32,
}

impl Actor for Ticker {
    type Message = ();

    fn handle(
        &mut self,
        _msg: (),
        ctx: &mut ActorContext,
    ) -> Result<(), ActorError> {
        if self.remaining == 0 {
            println!("Done!");
            return Ok(());
        }

        println!("Tick... {} remaining", self.remaining);
        self.remaining -= 1;

        // Schedule a message to self after 1 second
        ctx.schedule_timer(Duration::from_secs(1), ())
            .map_err(|e| ActorError::Handler(e.to_string()))?;
        Ok(())
    }
}

fn main() {
    let mut runtime = Runtime::new().unwrap();
    let id = runtime.spawn(Ticker { remaining: 3 }).unwrap();
    runtime.send(id, ().unwrap());
    runtime.shutdown();
}

```

This example uses: - **State** (`remaining` field) that persists across messages. - `ctx.schedule_timer` — sends the actor's own message type after a delay. - **Self-messaging** — the actor sends itself `()` messages on a timer.

The output would be:

```

Tick... 3 remaining
Tick... 2 remaining      (1 second later)
Tick... 1 remaining      (1 second later)
Done!                    (1 second later)

```

Chapter 4 covers the `Actor` trait in full detail, and Chapter 5 covers `ActorContext` — the methods you use to interact with the runtime from inside an actor. # Chapter 4: The Actor Trait

The `Actor` trait is the heart of Runact. It defines what an actor is and how the runtime interacts with it. This chapter covers the full trait, its design rationale, and common patterns.

## 1.19 4.1 The Trait

```
pub trait Actor: Send + 'static {
    type Message: Send + 'static;

    fn handle(
        &mut self,
        message: Self::Message,
        context: &mut ActorContext,
    ) -> Result<(), ActorError>;
}
```

Three items:

### 1.19.1 `Send + 'static` (trait bound)

The `Actor: Send + 'static` bound means every actor must be safe to move to another thread (`Send`) and must not hold references to local data (`'static`). This allows the scheduler to freely migrate actors between worker threads.

**What this implies:** - All fields must be `Send`. Common types like `String`, `Vec`, `HashMap` are `Send`. Types like `Rc<T>` and `RefCell<T>` are NOT `Send`. - No `&'a T` references with lifetimes (unless `'static`).

### 1.19.2 `type Message: Send + 'static`

The message type must also be `Send + 'static` so it can be safely transferred between threads through the mailbox channel.

### 1.19.3 `fn handle(&mut self, message, context) -> Result<(), ActorError>`

This is called for **every** message the actor receives. Key points:

- `&mut self` — you have exclusive access to your actor's state. Mutate it freely; no locks needed.
- `message` — **moved** into the handler. No cloning, no borrowing.
- `context` — your interface to the runtime (sending messages, spawning tasks, timers, etc.). See Chapter 5.
- Return `Ok(())` on success, `Err(ActorError::...)` on failure.

## 1.20 4.2 Returning Errors

The `ActorError` enum has three variants:

```
pub enum ActorError {
    Handler(String),    // General handler error
    Cancelled,          // Actor was cancelled
    Timeout,            // Operation timed out
    Panic(String),      // Actor panicked (set by runtime)
}
```

You construct `Handler(String)` for application-level errors:

```
fn handle(&mut self, msg: MyMessage, ctx: &mut ActorContext)
    -> Result<(), ActorError>
{
    if msg.value < 0 {
        return Err(ActorError::Handler(
            "value must be non-negative".to_string()
        ));
    }
    // process...
    Ok(())
}
```

When `handle` returns `Err`, the supervisor decides what to do. By default, `OneForOne` strategy restarts the actor with exponential backoff. The actor's `handle` is then called again for the next message in its mailbox.

**Note:** `handle` does NOT receive an `init` or `started` callback in the current API. Actors are initialized when spawned — just set up state in the constructor (the struct itself). If you need startup logic, send the actor a `Start` message as its first message.

## 1.21 4.3 A Complete Example: A Counter Actor

```
use runact::{Actor, ActorContext, ActorError};
use std::collections::HashMap;

/// Messages the Counter actor can receive.
#[derive(Debug)]
enum CounterMsg {
    /// Increment by `n` and return the new value.
    Increment(u64),
    /// Decrement by `n` and return the new value.
    Decrement(u64),
    /// Get the current value.
    Get,
}

/// A stateful counter actor.
struct Counter {
    value: u64,
}
```

```

impl Counter {
    fn new() -> Self {
        Counter { value: 0 }
    }
}

impl Actor for Counter {
    type Message = CounterMsg;

    fn handle(
        &mut self,
        msg: Self::Message,
        _ctx: &mut ActorContext,
    ) -> Result<(), ActorError> {
        match msg {
            CounterMsg::Increment(n) => {
                self.value += n;
                println!("Counter is now {}", self.value);
            }
            CounterMsg::Decrement(n) => {
                self.value = self.value.saturating_sub(n);
                println!("Counter is now {}", self.value);
            }
            CounterMsg::Get => {
                println!("Counter value is {}", self.value);
            }
        }
        Ok(())
    }
}

```

Key patterns shown: - **Enum messages** — A common Rust pattern for actors. Each variant is a different operation. The message enum itself carries the arguments. - **State in the struct** — value is private to the actor. No other actor can touch it directly. - **No blocking** — The handler runs to completion synchronously.

## 1.22 4.4 Stateful Actors

An actor's struct fields are its persistent state. Each time `handle` is called, `&mut self` gives you access to that state. After `handle` returns, the state remains as you left it — the actor hasn't been destroyed.

```

struct SessionManager {
    sessions: HashMap<ActorId, Session>,
}

impl Actor for SessionManager {

```

```

type Message = SessionEvent;

fn handle(&mut self, msg: SessionEvent, ctx: &mut ActorContext)
    -> Result<(), ActorError>
{
    match msg {
        SessionEvent::Login { user_id, actor_id } => {
            self.sessions.insert(actor_id, Session::new(user_id));
        }
        SessionEvent::Logout { actor_id } => {
            self.sessions.remove(&actor_id);
        }
        SessionEvent::GetUserCount(reply) => {
            let count = self.sessions.len();
            reply(count).map_err(|e| ActorError::Handler(e.to_string()))?;
        }
    }
    Ok(())
}

```

## 1.23 4.5 Multi-Message-Type Actors

Sometimes an actor needs to handle messages of different types. Rust’s trait system requires a single `Message` type, but you can use an enum:

```

enum WorkerMsg {
    Task(Task),
    Timeout(TimerId),
    Shutdown,
}

impl Actor for Worker {
    type Message = WorkerMsg;
    // ...
}

```

This is the standard approach. Avoid `Box<dyn Any>` — it defeats Rust’s type safety and is slower.

## 1.24 4.6 Actor Lifecycle

```

spawn(Actor)    [mailbox: empty]    message    handle()    Ok/Err
                                     (mailbox drains)
                                     no more msgs    idle (parked)

```

When an actor has no messages, it is **parked** — it consumes no scheduler resources. When a message arrives, the scheduler un parks one worker to process it. This is how Runact achieves massive actor counts (thousands or millions of idle actors) without performance cost.

When `handle` returns `Err(ActorError::Panic(...))`, the actor has panicked. The supervisor will restart it (if configured). When `Err(ActorError::Cancelled)` is returned, the actor is explicitly being shut down. # Chapter 5: Actor Context

`ActorContext` is the actor's interface to the runtime. It is passed as `&mut ActorContext` to every `handle` call. This chapter covers all the methods you need to: send messages, reply to requests, schedule timers, spawn async tasks, and offload compute work.

## 1.25 5.1 The Context API

---

```
pub struct ActorContext { /* private fields */ }

impl ActorContext {
    pub fn actor_id(&self) -> ActorId;

    pub fn send_to<M: Send + 'static>(
        &self, target: ActorId, message: M
    ) -> Result<(), RuntimeError>;

    pub fn reply<M: Send + 'static>(
        &self, message: M
    ) -> Result<(), RuntimeError>;

    pub fn is_request(&self) -> bool;

    pub fn spawn_compute<F, T>(
        &self, job: F
    ) -> Result<ComputeHandle<T>, ComputeError>
    where F: FnOnce() -> T + Send + 'static, T: Send + 'static;

    pub fn schedule_timer<M: Send + 'static>(
        &self, duration: Duration, message: M
    ) -> Result<TimerId, RuntimeError>;

    pub fn schedule_interval<M: Clone + Send + Sync + 'static>(
        &self, interval: Duration, message: M
    ) -> Result<TimerId, RuntimeError>;

    pub fn cancel_timer(&self, timer_id: TimerId);
}
```

## 1.26 5.2 Sending Messages to Other Actors

`ctx.send_to(target, message)` pushes a message onto another actor's mailbox. This is how actors talk to each other.

```

struct Router {
    worker: ActorId,
}

impl Actor for Router {
    type Message = RouteMsg;

    fn handle(&mut self, msg: RouteMsg, ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {
        match msg {
            RouteMsg::Request(data) => {
                // Forward to the worker actor
                ctx.send_to(self.worker, WorkMsg::Do(data))
                    .map_err(|e| ActorError::Handler(e.to_string()))?;
            }
            // ...
        }
        Ok(())
    }
}

```

### 1.26.1 send\_to vs runtime.send

| Method                             | Called from                            | Blocks?                          | Use case                        |
|------------------------------------|----------------------------------------|----------------------------------|---------------------------------|
| <code>ctx.send_to</code>           | Inside an actor handler                | No (uses <code>try_send</code> ) | Actor → actor messaging         |
| <code>runtime.send</code>          | Outside actors (main thread, external) | No (uses <code>try_send</code> ) | External → actor messaging      |
| <code>runtime.send_blocking</code> | Internal threads only                  | Yes (blocks until accepted)      | High-priority external messages |

## 1.27 5.3 Reply-to-Sender (Request/Reply)

When an actor receives a message via `Runtime::request` (not `send`), the runtime knows the caller expects a reply. `ctx.reply(message)` sends the reply back.

```

// External code sends a request
let request_handle = runtime.request(counter_id, CounterMsg::Get).unwrap();

// Inside the Counter actor:
fn handle(&mut self, msg: Self::Message, ctx: &mut ActorContext)
    -> Result<(), ActorError>
{
    match msg {
        CounterMsg::Get => {
            // ctx.is_request() is true here

```

```

        ctx.reply(self.value)
        .map_err(|e| ActorError::Handler(e.to_string()))?;
    }
    // ...
}
Ok(())
}

```

*// External code receives the reply*

```
let value: u64 = request_handle.recv().unwrap();
```

If the message was sent fire-and-forget (via `send`), `ctx.is_request()` returns `false` and `ctx.reply(...)` is a no-op.

## 1.28 5.4 Scheduling Timers

### 1.28.1 One-shot Timer

```
ctx.schedule_timer(Duration::from_secs(5), WakeUp {})?;
```

After 5 seconds, the actor receives `WakeUp {}`. The actor continues processing other messages during the wait — the timer is handled by the timer service, not by blocking the actor.

### 1.28.2 Periodic Timer (Interval)

```
ctx.schedule_interval(Duration::from_secs(10), Tick {})?;
```

The message must be `Clone + Send + Sync`. Each tick delivers a clone of the message. Returns a `TimerId` so you can cancel it later:

```
let timer_id = ctx.schedule_interval(Duration::from_secs(10), Tick {})?;
// ...
ctx.cancel_timer(timer_id); // stops the interval

```

### 1.28.3 Use Case: Heartbeat

```
struct Heartbeat {
    interval_id: Option<TimerId>,
}

```

```
impl Actor for Heartbeat {
    type Message = HeartMsg;

    fn handle(&mut self, msg: HeartMsg, ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {
        match msg {
            HeartMsg::Start => {
                self.interval_id = Some(
                    ctx.schedule_interval(Duration::from_secs(30), HeartMsg::Beat)?
                );
            }
        }
    }
}

```

```

        );
    }
    HeartMsg::Beat => {
        println!("Heartbeat at {:?}", std::time::Instant::now());
    }
    HeartMsg::Stop => {
        if let Some(id) = self.interval_id.take() {
            ctx.cancel_timer(id);
        }
    }
}
Ok(())
}
}

```

## 1.29 5.5 Offloading CPU-Bound Work

`ctx.spawn_compute(job)` runs a closure on the **compute pool** — a separate thread pool for CPU-intensive work. The actor is not blocked.

```

struct DataProcessor;

impl Actor for DataProcessor {
    type Message = ProcessFile;

    fn handle(&mut self, msg: ProcessFile, ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {
        let handle = ctx.spawn_compute(move || {
            // This runs on a compute worker thread
            expensive_hash_computation(&msg.data)
        })?;

        // Store the handle; check it later when the result is ready
        // (e.g., in response to a Poll message)
        self.pending.insert(msg.file_id, handle);
        Ok(())
    }
}

```

The `ComputeHandle<T>` lets you check if the work is done (see Chapter 11).

## 1.30 5.6 The `actor__id()` Method

```

let my_id = ctx.actor_id();
ctx.send_to(other_actor, ForwardMsg { from: my_id })?;

```

This is essential for: - Self-messaging (sending a message to yourself). - Registering with a registry

actor. - Including your identity in forwarded messages.

## 1.31 5.7 Important: Handle Must Return Quickly

ActorContext methods are all **non-blocking**. `send_to` uses `try_send` — if the target's mailbox is full, it returns `RuntimeError::MailboxFull` immediately. `spawn_compute` submits work to a bounded queue and returns immediately. `schedule_timer` just registers with the timer service.

This is by design: actors are cooperative. An actor must process one message, then yield back to the scheduler. Never block inside `handle`. # Chapter 6: Mailboxes and Backpressure

Every actor has a **mailbox** — a bounded queue that buffers incoming messages. This chapter explains mailbox mechanics, backpressure, and the reduction system that ensures fair scheduling.

## 1.32 6.1 What Is a Mailbox?

When you call `runtime.send(id, msg)` or `ctx.send_to(id, msg)`, the message is not delivered to the actor immediately. Instead, it is pushed onto the target actor's **mailbox** — a FIFO queue backed by a `crossbeam_channel::bounded` sender.

|        |         |                          |                        |
|--------|---------|--------------------------|------------------------|
| Sender | message | [mailbox: bounded queue] | Actor (when scheduled) |
|--------|---------|--------------------------|------------------------|

The actor processes messages one at a time, in order. While the actor is processing message N, messages N+1, N+2, ... wait in the queue.

## 1.33 6.2 Bounded Mailboxes

By default, mailboxes are **bounded**. The capacity is configurable via `RuntimeConfig`:

```
use runact::{Runtime, RuntimeConfig};

let config = RuntimeConfig {
    mailbox_capacity: 512, // default: 1024
    ..RuntimeConfig::default()
};

let mut runtime = Runtime::with_config(config).unwrap();
```

When the mailbox is full and you try to send:

```
match runtime.send(actor_id, message) {
    Ok(()) => { /* delivered */ }
    Err(RuntimeError::MailboxFull(id)) => {
        // The actor's mailbox is at capacity.
        // Options: retry, drop, or use send_blocking
    }
    Err(e) => { /* other error */ }
}
```

## 1.34 6.3 Backpressure

**Backpressure** is the mechanism by which a slow consumer signals a fast producer to slow down. Runact’s bounded mailboxes provide natural backpressure:

- If an actor processes messages slower than they arrive, the mailbox fills up.
- Once full, `send` returns `MailboxFull` immediately — the sender gets feedback and must handle it.
- This prevents unbounded memory growth.

### 1.34.1 Handling MailboxFull

```
// Retry with backoff
loop {
  match runtime.send(actor_id, msg.clone()) {
    Ok(()) => break,
    Err(RuntimeError::MailboxFull(_)) => {
      std::thread::sleep(Duration::from_millis(10));
      continue;
    }
    Err(e) => return Err(e),
  }
}
```

### 1.34.2 send vs send\_blocking

| Method                     | Behavior when full                           | Use case                                 |
|----------------------------|----------------------------------------------|------------------------------------------|
| <code>send</code>          | Returns <code>MailboxFull</code> immediately | Non-blocking, retry loop                 |
| <code>send_blocking</code> | Blocks until accepted                        | External threads that can afford to wait |

Inside actors, use `ctx.send_to` (always non-blocking). From external threads, choose `send` (fast-fail) or `send_blocking` (wait).

## 1.35 6.4 Reduction Counting

Runact’s scheduler uses **reduction counting** — a technique borrowed from BEAM. Each actor gets a budget of “reductions” (work units) per scheduling turn. When the budget is exhausted, the actor yields, and another actor gets scheduled.

### 1.35.1 How Reductions Work

```
Actor A: [msg1] → handle() → reduction budget-- → msg2 → handle() → budget exhausted
Scheduler: yield Actor A, schedule Actor B
Actor B: [msg3] → handle() → ...
```

The default budget is `MAX_REDUCTIONS` (typically a few hundred messages per turn). This ensures:

- **Fairness**: No single actor can monopolize a worker thread.
- **Responsiveness**: All actors

get a turn within a bounded time. - **Throughput**: Each actor processes multiple messages per scheduling turn (amortizing the context switch cost).

### 1.35.2 Why It Matters

Without reduction counting, a busy actor could process thousands of messages in a single scheduling turn, starving all other actors. With reductions, the runtime guarantees that  $N$  actors share  $N$  worker threads fairly, even if one actor is under heavy load.

## 1.36 6.5 Idle Actors

When an actor's mailbox is empty, the actor is **parked** — it is removed from the scheduler's run queue. Parked actors consume **zero** scheduler resources.

When a new message arrives:

```
Message arrives    mailbox (was empty)
```

```
Scheduler: actor is now "ready" - push onto run queue
```

```
Worker polls actor: handle(msg) for up to MAX_REDUCTIONS messages
```

This is why Runact can have hundreds of thousands of actors that spend most of their time idle — they cost nothing when parked, and wake up instantly when a message arrives.

## 1.37 6.6 Mailbox Internals

The mailbox is implemented as a `BoundedMailbox` struct (private to the runtime):

```
// Simplified - not public API
struct BoundedMailbox {
    sender: crossbeam_channel::Sender<MessageEnvelope>,
    receiver: crossbeam_channel::Receiver<MessageEnvelope>,
    capacity: usize,
}
```

Each actor has exactly one mailbox. The `ActorId`  $\rightarrow$  `sender` mapping is stored in the runtime's global registry:

```
// In Runtime
actors: Arc<RwLock<HashMap<ActorId, ActorInfo>>>,
senders: Arc<RwLock<HashMap<ActorId, crossbeam_channel::Sender<MessageEnvelope>>>>,
```

This is how `send` and `send_to` locate the target's mailbox without knowing the worker thread. #  
Chapter 7: Supervision

Supervision is Runact's fault-tolerance mechanism. A `Supervisor` manages a tree of child actors, restarting them when they crash according to a configurable `RestartStrategy`.

## 1.38 7.1 Why Supervision?

In distributed and long-running systems, failures are inevitable. Rather than letting a single actor crash bring down your entire application, supervision isolates failures and recovers automatically.

The Erlang/BEAM philosophy is: **let it crash**. Instead of defensive programming everywhere, you write your actors to fail fast on errors, and let the supervisor restart them in a known-good state.

## 1.39 7.2 RestartStrategy

```
pub enum RestartStrategy {
  OneForOne {
    max_restarts: usize,
    within: Duration,
    base_backoff: Duration,
  },
}
```

### 1.39.1 OneForOne

When a child actor fails, **only that child is restarted**. Siblings are unaffected. This is the simplest and most common strategy.

### 1.39.2 Parameters

| Field                     | Default | Description                                                             |
|---------------------------|---------|-------------------------------------------------------------------------|
| <code>max_restarts</code> | 5       | Maximum restarts within the <code>within</code> window before giving up |
| <code>within</code>       | 60s     | Time window for counting restarts                                       |
| <code>base_backoff</code> | 100ms   | Initial delay between restarts; doubles each time, capped at 30s        |

If a child crashes more than `max_restarts` times within `within`, the supervisor **escalates** — it fails itself, and its own supervisor (if any) must handle the failure.

### 1.39.3 Default Strategy

```
let strategy = RestartStrategy::default();
// OneForOne { max_restarts: 5, within: 60s, base_backoff: 100ms }
```

## 1.40 7.3 ChildSpec

A `ChildSpec` describes a child actor for the supervisor:

```
pub struct ChildSpec {
  pub name: String,
  pub restart_policy: RestartPolicy,
```

```
    pub shutdown_timeout: Duration,
}
```

### 1.40.1 RestartPolicy

| Policy    | Behavior                                               |
|-----------|--------------------------------------------------------|
| Permanent | Always restart on failure                              |
| Temporary | Never restart                                          |
| Transient | Restart only on abnormal exit (panic, not clean error) |

### 1.40.2 Builder Pattern

```
let child = ChildSpec::new("worker")
    .restart_policy(RestartPolicy::Permanent)
    .shutdown_timeout(Duration::from_secs(10));
```

## 1.41 7.4 Spawning a Supervisor

```
use runact::{
    ChildSpec, RestartPolicy, RestartStrategy, Runtime, Supervisor,
};

let mut runtime = Runtime::new().unwrap();

let strategy = RestartStrategy::default();
let children = vec![
    ChildSpec::new("worker-1"),
    ChildSpec::new("worker-2"),
];

let supervisor_id = runtime.spawn_supervisor(strategy, children).unwrap();
```

**Note:** `spawn_supervisor` creates the supervisor as a registered actor within the runtime. The supervisor's `ActorId` is returned so you can send it messages if needed. In the current implementation, the supervisor tracks child metadata; actual child actor spawning follows the patterns in Chapter 16 (Agent Servers).

## 1.42 7.5 How Restarts Work

Child panics in `handle()`

Supervisor detects failure

```
Check restart budget: count < max_restarts within window?
    Yes → wait base_backoff (*2^count), then restart child
```

No → escalate (fail supervisor)

### 1.42.1 Exponential Backoff

Restart 1: 100ms  
 Restart 2: 200ms  
 Restart 3: 400ms  
 Restart 4: 800ms  
 ...capped at 30s

This prevents rapid crash loops from overwhelming the system. If a child has a persistent bug, exponential backoff gives you time to notice and intervene before the system is consumed.

## 1.43 7.6 Supervised Worker Example

```
use runact::{Actor, ActorContext, ActorError};

struct Worker {
    config: WorkerConfig,
}

impl Actor for Worker {
    type Message = WorkItem;

    fn handle(&mut self, msg: WorkItem, ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {
        // If this panics, the supervisor will restart us.
        // Our state will be re-initialized as a fresh Worker.
        let result = process_item(&self.config, &msg);
        result.map_err(|e| ActorError::Handler(e.to_string()))
    }
}
```

When the supervisor restarts `Worker`, a new instance is created with fresh state. The old state (including any corrupted or inconsistent fields) is gone.

## 1.44 7.7 Supervisor as an Actor

In Runact’s current implementation, `Supervisor` is not itself an `Actor` trait implementor — it is managed by the runtime’s internal supervision machinery. The `Supervisor` struct tracks child specs and restart counts and is used by `spawn_supervisor`. Future versions will expose the supervisor as a first-class actor that can receive `HealthCheck` messages and dynamic child specifications.

## 1.45 7.8 Key Principles

1. **Isolation:** A crash in one child doesn’t affect siblings.
2. **Recovery:** Automatic restart with clean state.

3. **Rate limiting:** Too-fast restarts trigger backoff or escalation.
4. **Let it crash:** Don't catch every error — let the supervisor handle it. # Chapter 8: Actor-to-Actor Messaging

Actors communicate exclusively through messages. This chapter covers the two messaging patterns — fire-and-forget and request/reply — and the `Runtime` methods that support them.

## 1.46 8.1 Message Types

An actor's `Message` type defines its entire protocol. Use enums for multi-operation actors:

```
enum BankMsg {
    Deposit { amount: u64, to: AccountId },
    Withdraw { amount: u64, from: AccountId },
    Transfer { amount: u64, from: AccountId, to: AccountId },
    Balance(AccountId),
}

impl Actor for Bank {
    type Message = BankMsg;
    fn handle(&mut self, msg: BankMsg, ctx: &mut ActorContext)
        -> Result<(), ActorError> {
        match msg { /* ... */ }
        Ok(())
    }
}
```

The message type must be `Send + 'static`. Complex types are fine as long as all their fields are `Send + 'static`.

## 1.47 8.2 Fire-and-Forget: `send_to`

Fire-and-forget messages are delivered to the target's mailbox and the caller never waits for a reply. This is the most common pattern.

### 1.47.1 From Within an Actor

```
impl Actor for Router {
    type Message = RouteMsg;

    fn handle(&mut self, msg: RouteMsg, ctx: &mut ActorContext)
        -> Result<(), ActorError> {
    {
        ctx.send_to(self.worker, WorkMsg::Process(msg.data))
            .map_err(|e| ActorError::Handler(e.to_string()))?;
        Ok(())
    }
}
```

```

    }
}

```

`ctx.send_to` uses `try_send` — it returns immediately. If the target’s mailbox is full, it returns `RuntimeError::MailboxFull`.

### 1.47.2 From External Code

```

let mut runtime = Runtime::new().unwrap();
let worker_id = runtime.spawn(Worker::new()).unwrap();

```

```

// Send from outside the runtime
runtime.send(worker_id, WorkMsg::Start).unwrap();

```

`runtime.send` is also `try_send` — non-blocking. If you can afford to block until the mailbox accepts the message, use `runtime.send_blocking`:

```

runtime.send_blocking(worker_id, WorkMsg::Start).unwrap();

```

## 1.48 8.3 Request/Reply: `Runtime::request`

For request/reply, use `Runtime::request`. This sends a message with a one-shot reply channel and returns a `RequestHandle` for receiving the reply.

```

// External code sends a request
let handle = runtime.request(worker_id, Query::WhoAreYou)
    .expect("send request");

// Actor receives the request and replies
impl Actor for Worker {
    type Message = Query;
    fn handle(&mut self, msg: Query, ctx: &mut ActorContext)
        -> Result<(), ActorError> {
        match msg {
            Query::WhoAreYou => {
                assert!(ctx.is_request());
                ctx.reply("I am Worker 42".to_string());
            }
            _ => {}
        }
        Ok(())
    }
}

```

```

// External code receives the reply
let reply: Box<dyn Any + Send> = handle.recv().unwrap();
let name: String = *reply.downcast(String).unwrap();
println!("Reply: {}", name);

```

## 1.48.1 RequestHandle Methods

```

let handle = runtime.request(id, msg).unwrap();

// Block until reply
let reply = handle.recv()?;

// Try without blocking
match handle.try_recv() {
    Ok(reply) => { /* got reply */ }
    Err(_) => { /* no reply yet */ }
}

// Block with timeout
let reply = handle.recv_timeout(Duration::from_secs(5))?;

```

## 1.49 8.4 Actor-to-Actor Request/Reply

For request/reply where the **receiver is also an actor** (not external code), use `ctx.send_to` with a reply channel encoded in the message:

```

use std::sync::mpsc;

struct Ping;
struct Pong(mpsc::Sender<()>);

impl Actor for Service {
    type Message = Pong;
    fn handle(&mut self, msg: Pong, _ctx: &mut ActorContext)
        -> Result<(), ActorError> {
        println!("Received ping, sending pong");
        let _ = msg.0.send(());
        Ok(())
    }
}

impl Actor for Client {
    type Message = Ping;
    fn handle(&mut self, _msg: Ping, ctx: &mut ActorContext)
        -> Result<(), ActorError> {
        let (tx, rx) = mpsc::channel();
        ctx.send_to(self.service, Pong(tx))?;

        // Wait for reply (non-ideal: blocks actor handler)
        rx.recv_timeout(Duration::from_secs(5)).unwrap();
        Ok(())
    }
}

```

**Warning:** Blocking on a channel inside `handle` will block the scheduler worker thread. In production, store the `mpsc::Receiver` and check it in response to a timer message instead (see Chapter 11).

## 1.50 8.5 ActorId as a Communication Primitive

`ActorId` is `Copy`, `Clone`, `Hash`, `Eq`. This means you can use it as a key in a `HashMap` inside an actor:

```
struct RoomManager {
    members: HashMap<ActorId, UserInfo>,
}

impl Actor for RoomManager {
    type Message = RoomMsg;
    fn handle(&mut self, msg: RoomMsg, ctx: &mut ActorContext)
        -> Result<(), ActorError> {
        match msg {
            RoomMsg::Join { user_id, user_actor } => {
                self.members.insert(user_actor, user_id);
                // Broadcast to all existing members
                for &member in self.members.keys() {
                    ctx.send_to(member, RoomMsg::UserJoined(user_id))?;
                }
            }
            RoomMsg::Leave { user_actor } => {
                self.members.remove(&user_actor);
            }
            _ => {}
        }
        Ok(())
    }
}
```

## 1.51 8.6 Message Ordering

Messages from a single sender to a single target are delivered **in order** (because they go through a single channel). However, messages from **multiple senders** can interleave:

|          |      |           |                                        |
|----------|------|-----------|----------------------------------------|
| Sender A | msg1 |           |                                        |
|          |      | [mailbox] | Actor: handles msg1, then msg3         |
| Sender B | msg2 |           | (msg2 may arrive before or after msg1) |

If your actor needs global ordering, serialize through a single coordinator actor. If it needs causal ordering, include a sequence number or vector clock in your message. # Chapter 9: Async Tasks

Runact has a native async task executor for standard Rust `Futures`. This chapter covers how to spawn, await, and cancel async tasks.

## 1.52 9.1 spawn\_task

`Runtime::spawn_task` runs any `Future<Output = T> + Send + 'static` on `Runact`'s native executor:

```
use runact::Runtime;

let mut runtime = Runtime::new().unwrap();

let handle = runtime.spawn_task(async {
    // This runs on Runact's executor, not on an actor worker
    let result = some_async_operation().await;
    result
}).unwrap();

// Later, retrieve the result:
let result: String = handle.recv().unwrap();
```

### 1.52.1 Key Guarantees

1. **Pending futures consume no worker time.** When a future returns `Poll::Pending`, the executor parks it and polls something else. A `Runact`-specific `Waker` re-queues the task when the future becomes ready (e.g., when a timer fires or I/O completes).
2. **Panics are caught.** A panic inside a future is caught and surfaced to the `TaskHandle` as `TaskError::Panic` — it does NOT kill a worker thread.
3. **Detachable.** Dropping the `TaskHandle` detaches the task: it continues running to completion, but you can no longer retrieve the result.

## 1.53 9.2 TaskHandle

`spawn_task` returns a `TaskHandle<T>`:

```
pub struct TaskHandle<T> {
    pub fn id(&self) -> TaskId;
    pub fn recv(&self) -> Result<T, TaskError> where T: Clone + 'static;
    pub fn try_recv(&self) -> Option<Result<T, TaskError>> where T: Clone + 'static;
    pub fn recv_timeout(&self, timeout: Duration) -> Result<T, TaskError> where T: Clone + 'static;
}
```

### 1.53.1 recv — Block Until Done

```
let result = handle.recv()?; // blocks until task completes
```

### 1.53.2 try\_recv — Check Without Blocking

```
match handle.try_recv() {
    Some(Ok(result)) => { /* done */ }
    Some(Err(e)) => { /* error or panic */ }
```

```

    None => { /* still running */ }
}

```

### 1.53.3 recv\_timeout — Block With Timeout

```

match handle.recv_timeout(Duration::from_secs(5)) {
    Ok(result) => { /* got result */ }
    Err(TaskError::Timeout) => { /* too slow */ }
    Err(TaskError::Panic(msg)) => { /* task panicked */ }
    Err(TaskError::ExecutorShutdown) => { /* runtime shutting down */ }
}

```

### 1.53.4 Output Must Be Clone

The `TaskHandle` caches the result so multiple `recv()` calls return it. This requires `T: Clone`. If your output is not `Clone`, you must extract it on the first `recv()` call and not call it again.

## 1.54 9.3 Async Sleep and Timeout

Runact provides `async sleep` and `timeout` as free functions:

```

use runact::Runtime;
use std::time::Duration;

let handle = runtime.spawn_task(async {
    Runtime::sleep(Duration::from_secs(1)).await;
    let result = Runtime::timeout(
        Duration::from_secs(5),
        async_http_request()
    ).await;
    result
}).unwrap();

```

- `Runtime::sleep(d)` — yields the task for `d` duration.
- `Runtime::timeout(d, future)` — runs `future` with a deadline of `d`. Returns `Ok(output)` if it completes in time, or `Err(TaskError::Timeout)`.

## 1.55 9.4 Bridging Actors and Async Tasks

An actor can spawn an `async` task and receive the result via a timer-driven polling pattern:

```

use runact::{Actor, ActorContext, ActorError};
use std::time::Duration;

struct HttpWorker;

impl Actor for HttpWorker {
    type Message = HttpWork;
}

```

```

fn handle(&mut self, msg: HttpWork, ctx: &mut ActorContext)
    -> Result<(), ActorError>
{
    match msg {
        HttpWork::Fetch(url) => {
            // Spawn async task on the executor
            let handle = ctx.spawn(
                async move { fetch_url(&url).await }
            ).map_err(|e| ActorError::Handler(e.to_string()))?;

            // Poll via a timer
            let fetch_id = url;
            ctx.schedule_timer(Duration::from_millis(100), HttpWork::PollHandle { handle, fetch_id });
        }
        HttpWork::PollHandle { handle, fetch_id } => {
            match handle.try_recv() {
                Some(Ok(result)) => { /* process result */ }
                Some(Err(e)) => { /* handle error */ }
                None => {
                    // Still running - reschedule poll
                    ctx.schedule_timer(Duration::from_millis(100), msg)?;
                }
            }
        }
    }
    Ok(())
}

```

**Note:** `ctx.spawn` does not exist in the current API — async tasks are spawned via `runtime.spawn_task` from outside the actor, or the actor stores a `RuntimeSender` (see Chapter 16) to spawn tasks.

## 1.56 9.5 Task Groups and Cancellation

`TaskGroup` provides **structured concurrency** — a group of related tasks can be cancelled together:

```

use runact::{CancellationToken, TaskGroup};

// Each child gets a token derived from the parent
let parent_token = CancellationToken::new();
let child_token = parent_token.child_token();

let group = TaskGroup::new(parent_token);
group.spawn(async move {
    // Task checks child_token.is_cancelled() periodically
    // When parent_token.cancel() is called, child_token becomes cancelled

```

```
});
```

Cancellation is **cooperative** — tasks must check their token `is_cancelled()` and exit gracefully. Runact does NOT forcibly kill threads or futures. # Chapter 10: Timers

Timers let actors schedule messages to arrive after a delay. This chapter covers one-shot timers, periodic intervals, and the `TimerId` for cancellation.

## 1.57 10.1 One-shot Timer

`ctx.schedule_timer(duration, message)` sends a message to the actor after the specified delay:

```
use runact::{Actor, ActorContext, ActorError};
use std::time::Duration;

struct Delay;

impl Actor for Delay {
    type Message = ();

    fn handle(&mut self, _msg: (), ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {
        println!("Starting...");

        // Send myself () after 2 seconds
        ctx.schedule_timer(Duration::from_secs(2), ())
            .map_err(|e| ActorError::Handler(e.to_string()))?;

        println!("Timer scheduled - I am not blocked!");
        Ok(())
    }
}
```

Key properties: - **Non-blocking**: `schedule_timer` returns immediately. The actor is free to process other messages while the timer runs. - **The timer fires once**: After 2 seconds, the actor receives `()`. - **The message type** must be `Send + 'static` (same as actor messages). It does NOT need to be the actor's `Message` type — Runact can route any `Send + 'static` type through the timer service.

## 1.58 10.2 Periodic Timer (Interval)

`ctx.schedule_interval(interval, message)` sends a message repeatedly at fixed intervals:

```
struct Heartbeat;

impl Actor for Heartbeat {
    type Message = Beat;
```

```

fn handle(&mut self, msg: Beat, ctx: &mut ActorContext)
    -> Result<(), ActorError>
{
    match msg {
        Beat::Start => {
            println!("Heartbeat started");
            ctx.schedule_interval(Duration::from_secs(30), Beat::Pulse)
                .map_err(|e| ActorError::Handler(e.to_string()))?;
        }
        Beat::Pulse => {
            println!("Heartbeat at {:?}", std::time::Instant::now());
            // Pulse arrives every 30 seconds automatically
        }
    }
    Ok(())
}

#[derive(Clone)]
enum Beat { Start, Pulse }

```

### 1.58.1 Message Requirements

The interval message must be Clone + Send + Sync because it is cloned for each tick. String, u32, Vec<u8> all satisfy this.

### 1.58.2 Returns TimerId

```
let timer_id = ctx.schedule_interval(Duration::from_secs(30), Beat::Pulse)?;
```

## 1.59 10.3 TimerId and Cancellation

Both schedule\_timer and schedule\_interval return a TimerId. Store it to cancel the timer later:

```

struct Heartbeat {
    interval_id: Option<TimerId>,
}

impl Actor for Heartbeat {
    type Message = BeatMsg;

    fn handle(&mut self, msg: BeatMsg, ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {
        match msg {
            BeatMsg::Start => {

```

```

        self.interval_id = Some(
            ctx.schedule_interval(Duration::from_secs(30), BeatMsg::Pulse)?
        );
    }
    BeatMsg::Pulse => {
        println!("Pulse");
    }
    BeatMsg::Stop => {
        if let Some(id) = self.interval_id.take() {
            ctx.cancel_timer(id);
        }
    }
}
Ok(())
}
}

```

## 1.60 10.4 Drift Correction

For periodic timers, Runact applies **drift correction**: each tick is scheduled relative to the previous tick's target time, not the actual fire time. This means if a tick fires late (e.g., the actor was busy), subsequent ticks compensate to maintain the overall interval.

## 1.61 10.5 Runtime-Level Timers

The same timer service is accessible from the `Runtime` directly, for scheduling messages to actors from outside:

```

let timer_id = runtime.schedule_timer(
    Duration::from_secs(10),
    actor_id,
    MyMessage::Timeout,
);
// ...
runtime.cancel_timer(timer_id);

```

This is useful when setting up timers before an actor is spawned, or from external coordination code.

## 1.62 10.6 Use Cases

| Pattern          | Timer type | Example                            |
|------------------|------------|------------------------------------|
| Delayed response | one-shot   | Retry fetch after backoff          |
| Heartbeat        | interval   | Send ping every 30s                |
| TTL              | one-shot   | Expire a session after 1h          |
| Polling          | interval   | Check health every 5s              |
| Circuit breaker  | one-shot   | Open breaker for 60s after failure |

## Chapter 2

# Chapter 11: Compute Pool

The compute pool is a separate thread pool for CPU-intensive work. It ensures that heavy computations never block the scheduler workers that run actors and async tasks.

---

## 2.1 11.1 Why a Separate Pool?

Runact's scheduler uses a fixed number of worker threads (one per CPU core by default). These workers run both actor handlers and async task polls. If a single `handle` call takes 5 seconds to compute a SHA-256 hash over 10MB of data, that worker is stuck — and up to N-1 other workers may also be idle while waiting for that actor's messages.

The **compute pool** solves this: CPU-bound work is submitted to a separate pool of worker threads, and the actor is free to continue processing other messages.

|              |                              |                         |        |                   |
|--------------|------------------------------|-------------------------|--------|-------------------|
| Actor worker | <code>spawn_compute()</code> | Compute pool            | (busy) | Actor worker      |
| (free)       |                              | (dedicated CPU threads) |        | (receives result) |

## 2.2 11.2 `ctx.spawn__compute`

Inside an actor, use `ctx.spawn_compute`:

```
use runact::{Actor, ActorContext, ActorError};

struct Hasher;

impl Actor for Hasher {
    type Message = HashRequest;

    fn handle(&mut self, msg: HashRequest, ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {
        // This closure runs on the compute pool, not on the actor worker
        let handle = ctx.spawn_compute(move || {
```

```

        sha256_heavy_computation(&msg.data)
    })?;

    // Store the handle to poll later
    self.pending.insert(msg.request_id, handle);
    Ok(())
}
}

```

### 2.2.1 Requirements

- The closure must be `FnOnce() -> T + Send + 'static`.
- The return type `T` must be `Send + 'static`.
- No access to `&self` — the closure is moved to another thread.

## 2.3 11.3 ComputeHandle

`spawn_compute` returns a `ComputeHandle<T>`:

```

pub struct ComputeHandle<T> {
    pub fn cancel(&self);
    pub fn is_cancelled(&self) -> bool;
    pub fn try_recv(&self) -> Option<Result<T, ComputeError>> where T: 'static;
    pub fn recv(&self) -> Result<T, ComputeError> where T: 'static;
    pub fn recv_timeout(&self, timeout: Duration) -> Result<T, ComputeError> where T: 'static;
}

```

### 2.3.1 Polling for Results

```

// Check if done (non-blocking)
match handle.try_recv() {
    Some(Ok(result)) => { /* use result */ }
    Some(Err(e)) => { /* handle error */ }
    None => { /* still running - try again later */ }
}

// Wait for result (blocks - but only on compute pool thread if called from actor)
match handle.recv_timeout(Duration::from_secs(10)) {
    Ok(result) => { /* use result */ }
    Err(ComputeError::WorkerPanic(msg)) => { /* compute panicked */ }
    Err(ComputeError::SchedulerShutdown) => { /* runtime shutting down */ }
}

```

## 2.4 11.4 Cancellation

`ComputeHandle::cancel` sets a flag that the compute worker checks before executing the task:

```

let handle = ctx.spawn_compute(|| expensive_thing());

```

```
// If the actor receives a Cancel message before the task starts:
handle.cancel();

// Later, check:
if handle.is_cancelled() {
  // Task was cancelled, won't produce a result
}
```

Cancellation is **best-effort**: if the task is already running, `cancel` won't stop it — it only prevents tasks that haven't started yet. For long-running compute jobs, the closure should check an external cancellation flag periodically.

## 2.5 11.5 ComputeConfig

The compute pool is configured via `RuntimeConfig`:

```
use runact::{Runtime, RuntimeConfig, ComputeConfig};

let config = RuntimeConfig {
  compute: ComputeConfig {
    max_workers: 4,           // defaults to available_parallelism
    queue_capacity: 1024,     // bounded - backpressure when full
    task_timeout: None,
  },
  ..RuntimeConfig::default()
};

let mut runtime = Runtime::with_config(config).unwrap();
```

### 2.5.1 Queue Capacity and Backpressure

When the compute queue is full and you call `spawn_compute`, it returns `ComputeError::QueueFull`. The actor must handle this:

```
match ctx.spawn_compute(|| heavy_work()) {
  Ok(handle) => { self.pending.insert(id, handle); }
  Err(ComputeError::QueueFull) => {
    // Retry later or reject the request
  }
  Err(e) => {
    return Err(ActorError::Handler(e.to_string()));
  }
}
```

## 2.6 11.6 Use Cases for Compute Pool

| Task                                    | Why compute pool? |
|-----------------------------------------|-------------------|
| JSON encoding/decoding (large payloads) | CPU-bound, no I/O |
| Image/video processing                  | Heavy computation |

| Task                      | Why compute pool?              |
|---------------------------|--------------------------------|
| Cryptographic hashing     | CPU-intensive                  |
| Compression/decompression | CPU-bound                      |
| Parsing (JSON, CSV, etc.) | CPU-intensive for large inputs |
| Mathematical computation  | Pure CPU                       |
| Code indexing/search      | CPU-bound batch work           |

## 2.7 11.7 ComputePool vs AsyncTask

| Feature       | Compute pool ( <code>spawn_compute</code> ) | Async task ( <code>spawn_task</code> ) |
|---------------|---------------------------------------------|----------------------------------------|
| Runs on       | Compute worker threads                      | Async executor threads                 |
| Closure type  | <code>FnOnce() -&gt; T</code>               | <code>Future&lt;Output = T&gt;</code>  |
| Blocking OK?  | Yes (it's a dedicated thread)               | No (blocks worker)                     |
| Use case      | CPU-bound work                              | I/O-bound work                         |
| Result handle | <code>ComputeHandle&lt;T&gt;</code>         | <code>TaskHandle&lt;T&gt;</code>       |
| Cancellation  | <code>cancel()</code> (best-effort)         | <code>CancellationToken</code>         |

**Rule of thumb:** If it doesn't `await`, it goes to the compute pool. If it `awaits` on I/O, it goes to the async executor. # Chapter 12: TCP Runtime

Runact provides native TCP support via `runact::net::tcp_api`. This chapter covers the TCP listener, stream API, and how TCP integrates with the actor runtime.

## 2.8 12.1 Design Philosophy

Runact's TCP layer follows the design principle from `docs/runtime-networking-plan.md`:

TCP is the lowest-level network primitive.

Runact provides blocking-style TCP I/O (with non-blocking retries) that actors can use directly. Higher-level protocols (HTTP, WebSocket, TLS, DNS) are implemented in `runact-web` or application code.

### 2.8.1 Threading Model

Each `TcpStream` is non-blocking at the OS level, but the API provides blocking-style methods with internal retry loops. For concurrent handling, each connection typically gets its own thread (or actor + reader/writer threads).

## 2.9 12.2 TcpListener

```
use runact::net::tcp_api::TcpListener;
```

```
let listener = TcpListener::bind("127.0.0.1:8080");
```

### 2.9.1 accept()

```
loop {
    let client = listener.accept()?;
    // Handle client (spawn thread, spawn actor, etc.)
}
```

accept blocks until a connection arrives. Each call returns a `TcpStream` that is already set to non-blocking mode and registered with the I/O reactor.

### 2.9.2 local\_addr()

```
let addr = listener.local_addr()?;
println!("Listening on {}", addr);
```

## 2.10 12.3 TcpStream

```
use runact::net::tcp_api::TcpStream;

let mut stream = TcpStream::connect("127.0.0.1:8080");
```

### 2.10.1 Read

```
let mut buf = [0u8; 4096];
let n = stream.read(&mut buf)?; // retries on WouldBlock
```

read returns `Ok(0)` on connection close. It retries on `WouldBlock` internally (with a 1ms sleep), providing a blocking-style interface over a non-blocking socket.

### 2.10.2 write\_all + flush

```
stream.write_all(b"HTTP/1.1 200 OK\r\n\r\nHello")?;
stream.flush()?;
```

### 2.10.3 read\_exact

```
let mut header = [0u8; 8];
stream.read_exact(&mut header)?; // blocks until 8 bytes read
```

### 2.10.4 read\_to\_end

```
let mut buf = Vec::new();
stream.read_to_end(&mut buf)?; // reads until connection closes
```

read\_to\_end includes a heuristic: if it has read some data and then sees `WouldBlock` for 200+ consecutive attempts, it returns what it has rather than blocking forever.

### 2.10.5 shutdown

```
stream.shutdown(std::net::Shutdown::Both)?;
```

## 2.11 12.4 Simple Echo Server

```

use runact::net::tcp_api::TcpListener;
use std::io::{Read, Write};
use std::thread;

let listener = TcpListener::bind("127.0.0.1:8080")?;

loop {
    let mut client = listener.accept()?;

    thread::spawn(move || {
        let mut buf = [0u8; 4096];
        loop {
            match client.read(&mut buf) {
                Ok(0) => break, // connection closed
                Ok(n) => {
                    client.write_all(&buf[..n]).unwrap();
                    client.flush().unwrap();
                }
                Err(e) => {
                    eprintln!("Error: {}", e);
                    break;
                }
            }
        }
    });
}

```

## 2.12 12.5 TCP with Actors

The natural pattern is to spawn a per-connection actor, with separate reader/writer threads that feed the actor's mailbox:

```

use runact::{Actor, ActorContext, ActorError, Runtime};
use runact::net::tcp_api::TcpListener;
use std::io::Read;
use std::thread;

struct Connection {
    stream: TcpStream,
}

impl Actor for Connection {
    type Message = NetEvent;
    fn handle(&mut self, msg: NetEvent, _ctx: &mut ActorContext)
        -> Result<(), ActorError>
    {

```

```

    match msg {
      NetEvent::Data(data) => {
        // Process data, send response via writer thread
      }
      NetEvent::Closed => {
        // Cleanup
      }
    }
    Ok(())
  }
}

let mut runtime = Runtime::new()?;
let listener = TcpListener::bind("127.0.0.1:8080")?;

loop {
  let client = listener.accept()?;
  let conn_id = runtime.spawn(Connection { stream: client })?;
  // Spawn reader/writer threads...
}

```

This pattern — actor per connection with dedicated reader/writer threads — is how `runact-web`'s WebSocket server works (see Chapter 14).

## 2.13 12.6 I/O Reactor

Under the hood, `TcpListener` and `TcpStream` use an `epoll`-based `Reactor` (on Linux) that registers socket file descriptors for readiness events. This allows the runtime to efficiently multiplex many connections. # Chapter 13: HTTP with `runact-web`

`runact-web` provides minimal HTTP/1.1 support: `Request` and `Response` types, `Headers`, and a `Router` with path-parameter matching. This chapter shows how to build a simple HTTP server on top of `runact`'s TCP layer.

## 2.14 13.1 The Request Type

```

pub struct Request {
  pub method: Method,
  pub path: String,
  pub version: String,
  pub headers: Headers,
  pub body: Vec<u8>,
}

```

### 2.14.1 Parsing from Bytes

```
use runact_web::request::Request;
use runact::net::tcp_api::TcpListener;
use std::io::Read;
use std::thread;

let listener = TcpListener::bind("127.0.0.1:8080");

loop {
    let mut client = listener.accept()?;
    thread::spawn(move || {
        let mut buf = [0u8; 4096];
        let n = client.read(&mut buf).unwrap();
        let request = Request::parse(&buf[..n]).unwrap();
        println!("{}", request.method, request.path);
    });
}
```

### 2.14.2 Path Parameters

The router sets path parameters on the `Request` before handing it to the handler. You can read them with `request.param("name")`:

```
let id = request.param("id").unwrap_or("unknown");
```

## 2.15 13.2 The Response Type

```
pub struct Response {
    pub status: StatusCode,
    pub reason: String,
    pub version: String,
    pub headers: Headers,
    pub body: Vec<u8>,
}
```

### 2.15.1 Encoding

```
let bytes = response.encode();
client.write_all(&bytes)?;
client.flush()?;
```

## 2.16 13.3 The Router

```
use runact_web::router::Router;
use runact_web::request::Method;

let mut router = Router::new();
```

```

// Static path
router.route(Method::Get, "/", |req| {
    Response {
        status: StatusCode::OK,
        body: b"Welcome!".to_vec(),
        ..Default::default()
    }
});

// Path with parameter
router.route(Method::Get, "/users/:id", |req| {
    let id = req.param("id").unwrap_or("0");
    Response {
        status: StatusCode::OK,
        body: format!(r#"{"id": "{}}"#).to_string().into_bytes(),
        ..Default::default()
    }
});

// POST with body
router.route(Method::Post, "/users", |req| {
    Response {
        status: StatusCode::Created,
        body: req.body.clone(), // echo back
        ..Default::default()
    }
});

// 404 handler
router.not_found(|_req| {
    Response {
        status: StatusCode::NotFound,
        body: b"Not Found".to_vec(),
        ..Default::default()
    }
});

```

### 2.16.1 Router::handle

```
let response = router.handle(request);
```

Routes are matched in registration order (first match wins). Path patterns use **:name** for parameters: `/users/:id`, `/users/:user_id/posts/:post_id`.

## 2.17 13.4 Handler Trait

Any `Fn(Request) -> Response + Send + Sync + 'static` automatically implements the Handler trait. This means closures work directly:

```
router.route(Method::Get, "/health", |_| health_response());
```

## 2.18 13.5 Building a Complete HTTP Server

```
use runact::net::tcp_api::TcpListener;
use runact_web::headers::Headers;
use runact_web::request::{Method, Request};
use runact_web::response::{Response, StatusCode};
use runact_web::router::Router;
use std::io::{Read, Write};
use std::thread;

fn build_router() -> Router {
    let mut router = Router::new();
    router.route(Method::Get, "/health", |_| {
        Response {
            status: StatusCode::OK,
            reason: "OK".to_string(),
            version: "HTTP/1.1".to_string(),
            headers: Headers::new(),
            body: b"healthy".to_vec(),
        }
    });
    router
}

fn main() {
    let listener = TcpListener::bind("127.0.0.1:8080").unwrap();
    let router = build_router();

    loop {
        let mut client = listener.accept().unwrap();
        let mut buf = [0u8; 4096];
        let n = client.read(&mut buf).unwrap();
        let req = Request::parse(&buf[..n]).unwrap();
        let resp = router.handle(req);
        let bytes = resp.encode();
        client.write_all(&bytes).unwrap();
        client.flush().unwrap();
    }
}
```

## 2.19 13.6 IntoResponse

The `IntoResponse` trait converts values into `Response`:

```
impl IntoResponse for Response { ... }
impl IntoResponse for &str { ... }
impl IntoResponse for StatusCode { ... }
impl IntoResponse for (StatusCode, Vec<u8>) { ... }
```

This lets handlers return idiomatic types:

```
router.route(Method::Get, "/users/:id", |req| {
    let id = req.param("id").unwrap_or("0");
    (StatusCode::OK, format!(r#"{"id":"{id}"}"#).into_bytes())
});
```



## Chapter 3

# Chapter 14: WebSocket Server

`runact-web::websocket` provides a `WebSocketServer` that wraps any stream implementing `Read + Write + SetReadTimeout`, plus an `AsyncWebSocket` that handles the RFC 6455 protocol internals. This chapter covers the server API and how to build WebSocket applications.

---

### 3.1 14.1 WebSocketServer

```
use runact_web::websocket::server::WebSocketServer;
use runact_web::websocket::server::ServerEvent;

let server = WebSocketServer::bind("127.0.0.1:8080/ws")?;

server.accept_with_callback_and_config(
    |writer, event| {
        match event {
            ServerEvent::Connected => {
                println!("Client connected");
            }
            ServerEvent::Frame(frame) => {
                writer.send_text(&frame.payload_as_text()).unwrap();
            }
            ServerEvent::Closed => {
                println!("Client disconnected");
            }
            ServerEvent::Error(e) => {
                eprintln!("Error: {}", e);
            }
        }
    },
    WebSocketConfig { ping_interval: Some(Duration::from_secs(30)) },
)?;
```

### 3.1.1 ServerEvent

The callback receives `ServerEvent` variants:

| Variant               | When               | Description                              |
|-----------------------|--------------------|------------------------------------------|
| Connected             | Before reader loop | Client completed the WebSocket handshake |
| Frame(Frame)          | Each frame         | A parsed WebSocket frame                 |
| Closed                | Connection closed  | The peer closed the connection           |
| Error(HandshakeError) | Handshake failure  | Invalid or malformed upgrade request     |

### 3.1.2 WebSocketConfig

```
pub struct WebSocketConfig {
    pub ping_interval: Option<Duration>,
}
```

When `ping_interval` is set, a background thread sends Ping frames at the configured interval. The peer must respond with Pong frames (auto-handled by the `AsyncWebSocket` reader).

## 3.2 14.2 Stream Abstraction

`WebSocketServer` is generic over `S: Read + Write + SetReadTimeout + Send + 'static`. This allows it to work with both standard TCP streams and `runact`'s native TCP streams:

### 3.2.1 Standard TCP

```
let server = WebSocketServer::<std::net::TcpStream>::bind("127.0.0.1:8080/ws"?);
```

### 3.2.2 Runact TCP Bridge

```
use runact_web::websocket::runact_tcp::RunactTcpStream;
```

```
let server = WebSocketServer::<RunactTcpStream>::bind("127.0.0.1:8080/ws"?);
```

The `RunactTcpStream` adapter implements `Read`, `Write`, and `SetReadTimeout` (no-op, since `runact`'s TCP is non-blocking with internal retry loops).

## 3.3 14.3 Frame API

The `Frame` struct represents a single WebSocket frame:

```
pub struct Frame {
    pub fin: bool,           // Is this the final frame in a message?
    pub opcode: OpCode,      // Text, Binary, Ping, Pong, Close, Continuation
    pub masked: bool,        // Was the frame masked?
    pub mask_key: [u8; 4],   // Masking key (if masked)
    pub payload: Vec<u8>,    // Frame payload
}
```

### 3.3.1 OpCode

```
pub enum OpCode {
    Continuation,
    Text,
    Binary,
    Close,
    Ping,
    Pong,
}
```

## 3.4 14.4 ConnectionWriter

The ConnectionWriter passed to the callback is how you send data back to the client:

```
impl ConnectionWriter {
    pub fn send_text(&self, text: &str) -> Result<(), SendError>;
    pub fn send_binary(&self, data: &[u8]) -> Result<(), SendError>;
    pub fn send_ping(&self, data: &[u8]) -> Result<(), SendError>;
    pub fn send_pong(&self, data: &[u8]) -> Result<(), SendError>;
    pub fn send_close(&self, code: u16) -> Result<(), SendError>;
}
```

### 3.4.1 SendError

```
pub enum SendError {
    Full,      // Channel to writer thread is full
    Closed,    // Connection has been closed
}
```

## 3.5 14.5 Echo Server Example

```
use runact_web::websocket::server::{ServerEvent, WebSocketServer};
use runact_web::websocket::WebSocketConfig;
use std::time::Duration;

let server = WebSocketServer::bind("127.0.0.1:8080/ws"?);

server.accept_with_callback_and_config(
    |writer, event| {
        match event {
            ServerEvent::Connected => println!("Client connected"),
            ServerEvent::Frame(frame) => {
                // Echo back any text or binary frame
                if !frame.fin || matches!(frame.opcode, OpCode::Text | OpCode::Binary) {
                    let _ = writer.send_binary(&frame.payload);
                }
            }
        }
    }
)
```

```

        ServerEvent::Closed => println!("Client disconnected"),
        ServerEvent::Error(e) => eprintln!("Error: {}", e),
    }
},
WebSocketConfig { ping_interval: Some(Duration::from_secs(30)) },
)?;

```

### 3.6 14.6 Fragmentation Handling

The reader loop automatically reassembles fragmented messages per RFC 6455 §5.4:

- **First frame:** FIN=false, opcode = Text or Binary
- **Continuation frames:** FIN=false, opcode = Continuation
- **Final frame:** FIN=true, opcode = Continuation

Control frames (Ping, Pong, Close) can interleave with fragmented data — they are delivered immediately and never disrupt reassembly.

The reassembled frame is delivered with FIN=true and the original data opcode. See Chapter 15 for details.

### 3.7 14.7 Handshake

The WebSocket upgrade handshake is automatic:

1. Client sends HTTP request with Upgrade: websocket header and Sec-WebSocket-Key header.
2. Server validates the request and responds with HTTP/1.1 101 Switching Protocols and a computed Sec-WebSocket-Accept header.
3. The connection switches to the WebSocket framing protocol.

The `ServerEvent::Connected` variant fires **before** the reader loop starts, ensuring the application knows the connection is established. # Chapter 15: Message Fragmentation

Per RFC 6455 §5.4, a single WebSocket message can be split across multiple frames. This chapter explains how Runact's reader loop handles fragmentation.

---

### 3.8 15.1 Fragmentation Overview

Message: "Hello, world!"

Can be split into:

```

Frame 1: FIN=false, Opcode=Text,      Payload="Hello, "
Frame 2: FIN=false, Opcode=Continua, Payload="world"
Frame 3: FIN=true,  Opcode=Continua, Payload="!"

```

### 3.8.1 Rules (RFC 6455 §5.4)

1. The **first** frame of a message has `FIN=false` and a data opcode (`Text` or `Binary`).
2. **Subsequent** frames have `FIN=false` and opcode `Continuation`.
3. The **final** frame has `FIN=true` and opcode `Continuation`.
4. Control frames (`Ping`, `Pong`, `Close`) can be interleaved between any data frames.
5. A new `Text` or `Binary` frame while already in a fragmented message is a **protocol error**.

## 3.9 15.2 Runact's Reassembly

The reader loop in `AsyncWebSocket` maintains reassembly state:

```
// Simplified reader loop
let mut fragmented_opcode: Option<OpCode> = None;
let mut fragmented_payload: Vec<u8> = Vec::new();

while let Ok(frame) = Frame::parse(&buffer) {
    match frame.opcode {
        OpCode::Ping => { /* deliver immediately, don't affect reassembly */ }
        OpCode::Pong => { /* deliver immediately */ }
        OpCode::Close => { /* deliver immediately, break */ }
        OpCode::Text | OpCode::Binary => {
            if fragmented_opcode.is_some() {
                // Protocol error: new message started before old one finished
                return Err(ProtocolError);
            }
            if frame.fin {
                // Single-frame message - deliver immediately
                callback(Message::Frame(frame));
            } else {
                // Start of fragmented message
                fragmented_opcode = Some(frame.opcode);
                fragmented_payload = frame.payload;
            }
        }
        OpCode::Continuation => {
            if let Some(opcode) = fragmented_opcode {
                fragmented_payload.extend_from_slice(&frame.payload);
                if frame.fin {
                    // End of fragmented message - deliver reassembled
                    callback(Message::Frame(Frame {
                        fin: true,
                        opcode, // Original opcode (Text or Binary)
                        payload: std::mem::take(&mut fragmented_payload),
                        ..
                    }));
                    fragmented_opcode = None;
                }
            }
        }
    }
}
```

```

        // else: more continuation frames expected
    } else {
        // Protocol error: continuation without a started message
        return Err(ProtocolError);
    }
}
}
}
}

```

### 3.9.1 Key Design Points

1. **Control frames are delivered immediately.** They are never buffered as part of a fragmented message. This means a Ping arriving in the middle of a fragmented Text message is delivered as its own `Message` before the reassembled message.
2. **Reassembled frames preserve original metadata.** The delivered frame has `fin=true`, the original data opcode, and `masked=false` (server-side).
3. **Protocol errors break the connection.** If the rules above are violated, the reader loop delivers a `Message::Error` and stops.

## 3.10 15.3 Multi-Frame read() Processing

The reader also handles the case where multiple WebSocket frames arrive in a single TCP `read()` call:

```

TCP segment: [Frame1] [Frame2] [Frame3]
read() → [all bytes]

```

The reader loop processes frames sequentially, advancing an offset pointer through the buffer:

```

let mut offset = 0;
while offset < data.len() {
    let frame = Frame::parse(&data[offset..])?;
    offset += frame_encoded_size(&frame);
    // ... process frame ...
}

```

This ensures no frames are lost when the OS coalesces multiple WebSocket frames into one TCP segment.

### 3.10.1 frame\_encoded\_size

```

fn frame_encoded_size(frame: &Frame) -> usize {
    let mut size = 2; // header bytes
    let len = frame.payload.len();
    if len >= 126 {
        size += if len <= u16::MAX as usize { 2 } else { 8 };
    }
    if frame.masked { size += 4; } // mask key
    size += len;
}

```

```
    size
  }
```

### 3.11 15.4 Testing Fragmentation

The `ws_fragmentation.rs` integration test covers:

1. **Basic reassembly:** Client sends a 3-frame fragmented message. Server receives one reassembled frame with the full payload.
2. **Interleaved control frames:** Client sends a Ping frame in the middle of a fragmented Text message. Server receives the Ping (and auto-Pong response) immediately, then the reassembled Text message.

Both tests verify compliance with RFC 6455 §5.4. # Chapter 16: Agent Servers

This chapter walks through the `agent_server.rs` example: a WebSocket server that bridges run-act's TCP listener to a supervised `AgentActor` with a pluggable LLM adapter.

---

### 3.12 16.1 Architecture

WebSocket Client

(WebSocket over TCP)

WebSocketServer (runact-web)

(ServerEvent: Frame)

ConnectionWriter callback

(RuntimeSender::send)

AgentActor (runact)

ModelAdapter (trait)

MockAdapter (example)

ActorContext

spawn\_compute (CPU work)

schedule\_timer (delays)

send\_to (inter-actor messaging)

### 3.13 16.2 The Agent Message Protocol

```
enum AgentMessage {
  UserInput(String),           // User sent a message
  LLMResponse(String),         // Adapter returned a response
  Error(String),               // Error from the adapter
}
```

### 3.14 16.3 The ModelAdapter Trait

```
trait ModelAdapter: Send + 'static {
  fn generate(&self, prompt: &str) -> String;
}
```

This is the pluggable interface. The example provides a `MockAdapter` that simulates an LLM:

```
struct MockAdapter;

impl ModelAdapter for MockAdapter {
  fn generate(&self, prompt: &str) -> String {
    format!("Mock response to: {}", prompt)
  }
}
```

### 3.15 16.4 The AgentActor

```
struct AgentActor {
  adapter: Box<dyn ModelAdapter>,
  runtime_sender: RuntimeSender,
}

impl Actor for AgentActor {
  type Message = AgentMessage;

  fn handle(&mut self, msg: AgentMessage, ctx: &mut ActorContext)
    -> Result<(), ActorError>
  {
    match msg {
      AgentMessage::UserInput(input) => {
        // Offload LLM call to compute pool
        let sender = self.runtime_sender.clone();
        let conn_id = ctx.actor_id();
        ctx.spawn_compute(move || {
          let response = self.adapter.generate(&input);
          // Forward response back to this actor
          sender.send(conn_id, AgentMessage::LLMResponse(response))
            .ok();
        })?;
      }
      AgentMessage::LLMResponse(response) => {
        // Send response to the WebSocket client
        // (The WebSocket callback sends this to the actor,
        //  and here we'd forward it back through a writer thread)
        println!("Agent response: {}", response);
      }
      AgentMessage::Error(e) => {
```

```

        eprintln!("Agent error: {}", e);
    }
}
Ok(())
}
}

```

## 3.16 16.5 Bridging WebSocket to Actors

Each WebSocket connection gets its own `AgentActor` instance:

```

let server = WebSocketServer::bind("127.0.0.1:8080/ws"?);
let runtime_sender = runtime.sender();

server.accept_with_callback_and_config(
    |writer, event| {
        match event {
            ServerEvent::Connected => {
                // Spawn a new AgentActor for this connection
                // Note: Actor spawning requires &mut Runtime, so
                // for external threads, use RuntimeSender for messaging only.
                // In this example, the agent is pre-spawned.
            }
            ServerEvent::Frame(frame) => {
                // Forward the frame's text payload to the actor
                let text = String::from_utf8_lossy(&frame.payload);
                runtime_sender.send(agent_id, AgentMessage::UserInput(text.to_string()));
            }
            ServerEvent::Closed => { /* cleanup */ }
            ServerEvent::Error(e) => { /* handle */ }
        }
    },
    WebSocketConfig::default(),
)?;

```

### 3.16.1 Key Patterns

1. **RuntimeSender** is `Clone` — pass it to each connection thread.
2. **Per-connection actors** — each WebSocket client gets its own `AgentActor` with isolated state.
3. **Compute pool offloading** — LLM calls run on compute workers, not the scheduler.
4. **Non-blocking communication** — `RuntimeSender::send` is `try_send`; the actor processes messages from its mailbox.

## 3.17 16.6 Full Example

See `examples/agent_server.rs` for the complete implementation. The example:

```

fn main() {
    let mut runtime = Runtime::new().unwrap();
    let sender = runtime.sender();

    let server = WebSocketServer::bind("127.0.0.1:8080/ws").unwrap();

    server.accept_with_callback_and_config(
        |writer, event| { /* forward messages to actor */ },
        WebSocketConfig { ping_interval: Some(Duration::from_secs(30)) },
    ).unwrap();
}

```

### 3.18 16.7 Testing

The `ws_agent_server.rs` integration test verifies: - WebSocket handshake succeeds (HTTP 101 response) - Message forwarding from client → actor works end-to-end

### 3.19 16.8 Production Considerations

- Replace `MockAdapter` with a real LLM client (OpenAI, Anthropic, etc.)
- Add message history and context window management
- Add rate limiting and token budgeting
- Use `supervisor` for automatic restart on crash
- Add graceful shutdown for in-flight requests # Chapter 17: REST APIs

This chapter walks through the `agent_api.rs` example: a REST API server using `runact-web`'s Router with `runact`'s TCP listener.

---

### 3.20 17.1 Architecture

HTTP Client

(HTTP/1.1 over TCP)

TcpListener (`runact::net`)

(accept each connection)

Thread per connection

(read HTTP request)

`Router::handle(Request) → Response`

TcpStream (write HTTP response)

## 3.21 17.2 The Router

The Router matches HTTP requests to handlers based on method and path patterns:

```
use runact_web::router::Router;
use runact_web::request::{Method, Request};
use runact_web::response::{Response, StatusCode};

let mut router = Router::new();

router.route(Method::Get, "/api/agents", |req| {
    Response {
        status: StatusCode::OK,
        body: b"[]".to_vec(),
        ..Default::default()
    }
});

router.route(Method::Post, "/api/agents", |req| {
    // Create an agent
    Response {
        status: StatusCode::Created,
        body: req.body.clone(),
        ..Default::default()
    }
});

router.route(Method::Get, "/api/agents/:id", |req| {
    let id = req.param("id").unwrap_or("0");
    Response {
        status: StatusCode::OK,
        body: format!(r#"{"id":"{id}"}"#).into_bytes(),
        ..Default::default()
    }
});
```

### 3.21.1 Path Parameters

Routes can capture path segments with `:name`:

```
router.route(Method::Get, "/api/agents/:id/messages/:msg_id", |req| {
    let agent_id = req.param("id").unwrap();
    let msg_id = req.param("msg_id").unwrap();
    // ...
});
```

### 3.22 17.3 Shared State

For a REST API server, you need shared state (e.g., a session store). Since each HTTP connection runs on its own thread, use `Arc<Mutex<...>>`:

```
use std::sync::{Arc, Mutex};

struct SessionStore {
    sessions: Mutex<Vec<Session>>,
}

struct Session {
    id: u64,
    name: String,
}

let store = Arc::new(SessionStore {
    sessions: Mutex::new(Vec::new()),
});

// Pass `Arc::clone(&store)` to each connection thread
let store_clone = Arc::clone(&store);
let response = router.handle(request);
```

**Note:** In a production system, this shared state would be an actor managed by the runact runtime (see Chapter 16). The `Arc<Mutex>` pattern here is for the minimal example. Using an actor gives you message-based concurrency, supervision, and mailbox backpressure.

### 3.23 17.4 Reading HTTP Requests

HTTP requests are parsed from raw TCP bytes:

```
use runact_web::request::Request;
use std::io::Read;

let mut buf = [0u8; 4096];
let n = client.read(&mut buf)?;
let request = Request::parse(&buf[..n])?;
```

`Request::parse` handles: - Start line (method, path, version) - Headers (including `Content-Length`)  
- Body extraction

### 3.24 17.5 The Full Server Loop

```
use runact::net::tcp_api::TcpListener;
use runact_web::router::Router;
use std::io::{Read, Write};
use std::sync::{Arc, Mutex};
```

```

use std::thread;

fn main() {
    let store = Arc::new(SessionStore::new());
    let router = build_router(Arc::clone(&store));

    let listener = TcpListener::bind("127.0.0.1:8080").unwrap();

    loop {
        let mut client = listener.accept().unwrap();
        let router = Arc::new(router);

        thread::spawn(move || {
            let mut buf = [0u8; 4096];
            let n = client.read(&mut buf).unwrap();
            let request = Request::parse(&buf[..n]).unwrap();
            let response = router.handle(request);
            let bytes = response.encode();
            client.write_all(&bytes).unwrap();
            client.flush().unwrap();
        });
    }
}

```

### 3.24.1 Why Thread-Per-Connection?

This is the simplest possible HTTP server. For production use, you would instead:

1. Use the **actor model** — spawn a per-connection actor for each TCP connection.
2. Use **async tasks** — spawn an async task per connection that uses `runact::Runtime::sleep` and async I/O.
3. Use an **actor-per-connection** pattern with reader/writer threads, just like the WebSocket server (Chapter 14).

## 3.25 17.6 JSON Helpers

Since `runact-web` has no `serde` dependency, JSON is constructed manually:

```

fn json_response(status: StatusCode, body: &str) -> Response {
    Response {
        status,
        reason: status.reason().to_string(),
        version: "HTTP/1.1".to_string(),
        headers: Headers::new(),
        body: body.as_bytes().to_vec(),
    }
}

```

For more complex JSON, consider adding `serde_json` as a dependency in your application (`runact-`

web core stays dependency-free).

## 3.26 17.7 Testing

The `ws_agent_api.rs` integration test verifies the full REST lifecycle:

```
// GET /api/health → 200 "ok"
// POST /api/agents → 201 (creates session)
// GET /api/agents → 200 (lists sessions)
// GET /api/agents/:id → 200 (get single session)
// DELETE /api/agents/:id → 200 (delete session)
// GET /api/agents/:id (deleted) → 404
```

See `tests/ws_agent_api.rs` for the test implementation. # Chapter 18: API Reference

This chapter is a quick-reference guide to Runact's public API. For detailed explanations, see the chapters in Parts I–V.

---

## 3.27 18.1 runact::Runtime

```
pub struct Runtime {
    pub fn new() -> Result<Self, RuntimeError>;
    pub fn with_config(config: RuntimeConfig) -> Result<Self, RuntimeError>;

    pub fn spawn<A: Actor>(&mut self, actor: A) -> Result<ActorId, RuntimeError>;
    pub fn spawn_supervisor(
        &self, strategy: RestartStrategy, children: Vec<ChildSpec>
    ) -> Result<ActorId, RuntimeError>;

    pub fn send<M: Send + 'static>(
        &self, target: ActorId, message: M
    ) -> Result<(), RuntimeError>;
    pub fn send_blocking<M: Send + 'static>(
        &self, target: ActorId, message: M
    ) -> Result<(), RuntimeError>;

    pub fn request<M: Send + 'static>(
        &self, target: ActorId, message: M
    ) -> Result<RequestHandle, RuntimeError>;

    pub fn spawn_task<F, T>(
        &self, future: F
    ) -> Result<TaskHandle<T>, TaskError>
    where F: Future<Output = T> + Send + 'static, T: Send + 'static;

    pub fn schedule_timer<M: Send + 'static>(
        &self, duration: Duration, target: ActorId, message: M
```

```

    ) -> TimerId;
    pub fn cancel_timer(&self, timer_id: TimerId);

    pub fn sender(&self) -> RuntimeSender;
    pub fn shutdown(&mut self) -> Result<(), RuntimeError>;

    pub fn list_actors(&self) -> Result<Vec<ActorInfo>, RuntimeError>;
    pub fn inspect_actor(&self, id: ActorId) -> Result<Option<ActorInfo>, RuntimeError>;

    pub fn compute(&self) -> &ComputeScheduler;
}

```

### 3.28 18.2 runact::RuntimeSender

```

pub struct RuntimeSender {
    pub fn send<M: Send + 'static>(
        &self, target: ActorId, message: M
    ) -> Result<(), RuntimeError>;
    pub fn spawn<A: Actor>(&self, actor: A) -> Result<ActorId, RuntimeError>;
}

```

RuntimeSender is Clone. It can be sent to any thread.

### 3.29 18.3 Actor Types

```

pub trait Actor: Send + 'static {
    type Message: Send + 'static;
    fn handle(
        &mut self,
        message: Self::Message,
        context: &mut ActorContext,
    ) -> Result<(), ActorError>;
}

pub struct ActorContext {
    pub fn actor_id(&self) -> ActorId;
    pub fn send_to<M: Send + 'static>(
        &self, target: ActorId, message: M
    ) -> Result<(), RuntimeError>;
    pub fn reply<M: Send + 'static>(
        &self, message: M
    ) -> Result<(), RuntimeError>;
    pub fn is_request(&self) -> bool;
    pub fn spawn_compute<F, T>(
        &self, job: F
    ) -> Result<ComputeHandle<T>, ComputeError>
    where F: FnOnce() -> T + Send + 'static, T: Send + 'static;
}

```

```

pub fn schedule_timer<M: Send + 'static>(
    &self, duration: Duration, message: M
) -> Result<TimerId, RuntimeError>;
pub fn schedule_interval<M: Clone + Send + Sync + 'static>(
    &self, interval: Duration, message: M
) -> Result<TimerId, RuntimeError>;
pub fn cancel_timer(&self, timer_id: TimerId);
}

pub struct ActorId(u64);
impl ActorId {
    pub fn new(id: u64) -> Self;
    pub fn as_u64(self) -> u64;
}

pub enum ActorError {
    Handler(String),
    Cancelled,
    Timeout,
    Panic(String),
}

```

### 3.30 18.4 Error Types

```

pub enum RuntimeError {
    ActorNotFound(ActorId),
    MailboxFull(ActorId),
    RuntimeStopped,
    InvalidConfig(String),
}

pub enum TaskError {
    Panic(String),
    ExecutorShutdown,
    Timeout,
}

pub enum ComputeError {
    SchedulerShutdown,
    QueueFull,
    WorkerPanic(String),
}

```

### 3.31 18.5 Config Types

```

pub struct RuntimeConfig {
    pub compute: ComputeConfig,
}

```

```

    pub mailbox_capacity: usize,
    pub shutdown_timeout: Duration,
}

pub struct ComputeConfig {
    pub max_workers: usize,
    pub queue_capacity: usize,
    pub task_timeout: Option<Duration>,
}

```

### 3.32 18.6 runact-web Public API

```

// HTTP
pub mod request::{Method, Request};
pub mod response::{Response, StatusCode};
pub mod headers::{Headers};
pub mod router::{Router, IntoResponse};

// WebSocket
pub mod websocket::{
    frame::{Frame, OpCode},
    async_ws::{AsyncWebSocket, AsyncWriter, WebSocketConfig, Message, SendError},
    server::{WebSocketServer, ServerEvent, HandshakeError, ConnectionWriter},
    upgrade::{validate_upgrade_request, build_upgrade_response},
};

```

### 3.33 18.7 Task Types

```

pub struct TaskHandle<T> {
    pub fn id(&self) -> TaskId;
    pub fn recv(&self) -> Result<T, TaskError> where T: Clone + 'static;
    pub fn try_recv(&self) -> Option<Result<T, TaskError>> where T: Clone + 'static;
    pub fn recv_timeout(&self, timeout: Duration) -> Result<T, TaskError> where T: Clone + 'static;
}

pub struct ComputeHandle<T> {
    pub fn cancel(&self);
    pub fn is_cancelled(&self) -> bool;
    pub fn try_recv(&self) -> Option<Result<T, ComputeError>> where T: 'static;
    pub fn recv(&self) -> Result<T, ComputeError> where T: 'static;
    pub fn recv_timeout(&self, timeout: Duration) -> Result<T, ComputeError> where T: 'static;
}

pub struct CancellationToken {
    pub fn is_cancelled(&self) -> bool;
    pub fn cancel(&self);
    pub fn child_token(&self) -> CancellationToken;
}

```

```
}
```

## Chapter 4

# Chapter 19: Invariants

This chapter aggregates the runtime and networking invariants that all Runact code must uphold. These are enforced by tests, clippy lints, and the architecture itself.

---

### 4.1 19.1 Async Runtime Invariants

From docs/async-runtime.md §22:

1. **Pending tasks consume no worker time.** A future that returns `Poll::Pending` is parked until its Waker fires. The worker moves on to another task.
2. **CPU-heavy work never runs on async/actor workers.** CPU-bound code must be submitted to `ctx.spawn_compute`, which runs on the dedicated compute pool.
3. **Actors never block a worker waiting for I/O.** An actor's `handle` must return quickly. Long-running operations (HTTP requests, file I/O) must be offloaded to async tasks or the compute pool.
4. **Task failure never terminates the runtime.** A panic in an async task is caught by the executor and surfaced as `TaskError::Panic` on the `TaskHandle`.
5. **Cancellation is cooperative.** A `CancellationToken` signals cancellation, but the task must check `is_cancelled()` and exit gracefully. The runtime does not force-kill threads or futures.
6. **Shutdown deterministically terminates owned tasks.** When `Runtime::shutdown` is called, all spawned tasks, actors, and compute jobs are cancelled or joined with a timeout. The caller does not return until this is complete.
7. **Actors and async tasks remain fairly scheduled.** Reduction counting and task budgets ensure neither actors nor async tasks can starve each other.

### 4.2 19.2 Networking Invariants

From docs/runtime-networking-plan.md §30:

1. **Runact is NOT an HTTP/WebSocket/TLS/AI framework.** The core only provides TCP, process management, and I/O readiness. HTTP, WebSocket, TLS, and protocol handling live in `runact-web` or application code.
2. **TCP is the lowest-level network primitive.** All networking in runact core is socket-based TCP. Higher-level protocols are layered on top.
3. **Actors and async tasks remain distinct.** Actors communicate via messages; async tasks return values. They do not merge into a single abstraction.
4. **Structured concurrency prevents orphaned tasks.** All spawned async tasks are reachable via `TaskHandle` for the lifetime of the runtime. `TaskGroup` enables scoped cancellation.
5. **Process cancellation prevents orphaned child processes.** When a process actor is cancelled, its child processes are also terminated.
6. **Tokio is optional, not fundamental.** Runact's executor is native. Applications can use Tokio for specific I/O libraries, but the core does not depend on it.
7. **Runact must remain useful independently of PaperOS.** The runtime must be usable for general-purpose applications, not just the PaperOS editor.

### 4.3 19.3 Hard Rules (from AGENTS.md)

1. **No new dependencies** (core deps: `crossbeam-channel`, `serde`, `tracing`, `thiserror`).
2. **No Tokio in core.**
3. **No type-error suppression** (as `any` / `unsafe` / `.unwrap()` in library code).
4. **Every public type is `#[must_use]`.**
5. **No `println!`/`eprintln!` in library code** — use `tracing`.
6. **Error types use `thiserror`.**
7. **MSRV 1.85** — no language features newer than 1.85.

### 4.4 19.4 Verification

All invariants are verified by: - Integration tests in `tests/` and `runact-web/tests/` - `cargo clippy --all-targets -- -D warnings` - `RUSTDOCFLAGS="-D warnings" cargo doc --no-deps` - The `verify-rust.sh` script (if present)

To run the full gate:

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --all-targets
cargo test --doc
RUSTDOCFLAGS="-D warnings" cargo doc --no-deps
```

## Chapter 5

# Chapter 20: Architectural Decisions

This chapter summarizes the key architectural decisions documented in the ADR series ([docs/adr/](#)).

---

### 5.1 20.1 ADR-0001: BEAM-Style Scheduler

**Decision:** Use a BEAM-style scheduler with work-stealing.

**Context:** Runact needs to run thousands of lightweight actors concurrently across N worker threads.

**Decision:** Each worker maintains a run queue. When a worker is idle, it steals work from another worker's queue. Actors are scheduled using reduction counting (a budget of work per turn).

**Consequences:** - Fair scheduling across all actors. - Low overhead for idle actors (parked, no polling). - Natural scaling with CPU cores.

### 5.2 20.2 ADR-0002: Dual Scheduler Architecture

**Decision:** Separate actor scheduler from async task executor, but share worker threads.

**Context:** Actors and async tasks have different scheduling requirements. Actors need reduction-based fairness; async tasks need Waker-driven rescheduling.

**Decision:** Both run on the same pool of worker threads, but each has its own run queue and scheduling policy. Actors use reduction counting; async tasks use the standard `Poll/Waker` mechanism.

**Consequences:** - Shared worker pool avoids thread overhead. - Isolation prevents one type from starving the other. - Simplification: no separate thread pools to manage.

### 5.3 20.3 ADR-0003: Native Async Runtime (No Tokio)

**Decision:** Implement a native Future executor. Do not depend on Tokio.

**Context:** Tokio is the dominant Rust async runtime, but it is heavy and opinionated. Runact's core should be self-contained.

**Decision:** Implement `Future` polling, `Waker`, `Context`, `Poll` directly. Provide `spawn_task`, `sleep`, and `timeout` APIs that mirror Tokio's surface but use Runact's own scheduler.

**Consequences:** - No Tokio dependency in core. - Can interoperate with Tokio at the application layer if needed. - Full control over scheduling semantics (reduction budgeting).

## 5.4 20.4 Additional Decisions

### 5.4.1 Compute Pool Separation

**Decision:** CPU-bound work goes to a separate compute pool, never on scheduler workers.

**Rationale:** A CPU-bound `handle` call would block the worker, starving all other actors on that thread.

### 5.4.2 Capability-Based Security

**Decision:** Resources are accessed via unforgeable capabilities (`Capability<T>`, `ResourceHandle<T>`), not string names.

**Rationale:** The actor model's isolation model aligns with capability-based security. Only actors holding a capability can access the resource.

### 5.4.3 Cooperative Cancellation

**Decision:** Cancellation is cooperative — tasks check `CancellationToken::is_cancelled()` and exit gracefully.

**Rationale:** Force-killing threads is unsafe in Rust (no destructors run). Cooperative cancellation ensures clean state.